Analysis of Unstructured Data

Lecture 3 — The Mathematical Toolkit: Similarity, High-Dimensional Geometry, Dimensionality Reduction

Janusz Szwabiński

Outlook

1. A motivating example
2. Mathematical foundations
   - Inner products and similarity measures (cosine, Euclidean, Mahalanobis)
   - The curse of dimensionality (volume, distance concentration, near-orthogonality)
   - Random projections and the Johnson–Lindenstrauss lemma
   - PCA: variance maximisation, covariance eigendecomposition and the SVD
   - Randomised PCA at scale (CPU / CUDA / MPS)
   - Neighbourhood-preserving maps: t-SNE (implemented from scratch) and UMAP
3. Real-world examples
   - Low-rank image compression
   - Anomaly detection by reconstruction error
   - How *not* to read a t-SNE plot

Reading

1. Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning*, ch. 14
2. Géron, *Hands-On Machine Learning*, ch. 8
3. Halko, Martinsson & Tropp, *Finding structure with randomness*, SIAM Review 53 (2011)
4. van der Maaten & Hinton, *Visualizing data using t-SNE*, JMLR 9 (2008)
5. McInnes, Healy & Melville, *UMAP*, arXiv:1802.03426

In [1]:
```
import time
import warnings

import numpy as np
import matplotlib.pyplot as plt
from scipy.special import gammaln
from sklearn.datasets import load_digits, load_sample_image
from sklearn.decomposition import PCA

warnings.filterwarnings("ignore", category=FutureWarning)
rng = np.random.default_rng(0)
plt.rcParams["figure.dpi"] = 110
```

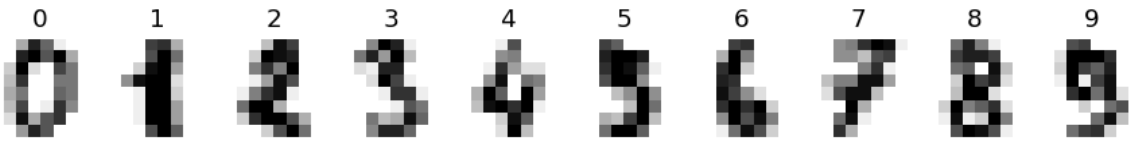
1. Sixty-four pixels, but how many numbers?

The `digits` dataset bundled with scikit-learn (a public UCI set) holds 1797 handwritten digits, each an 8×8 grey-scale image. As vectors they live in $\mathbb{R}^{64}$.

In [2]:
```
digits = load_digits()
X, y = digits.data, digits.target          # X: (1797, 64), pixel values 0..16

fig, axes = plt.subplots(1, 10, figsize=(10, 1.4))
for ax, img, lab in zip(axes, digits.images, digits.target):
    ax.imshow(img, cmap="gray_r")
    ax.set_title(int(lab))
    ax.axis("off")
plt.show()

X.shape
```



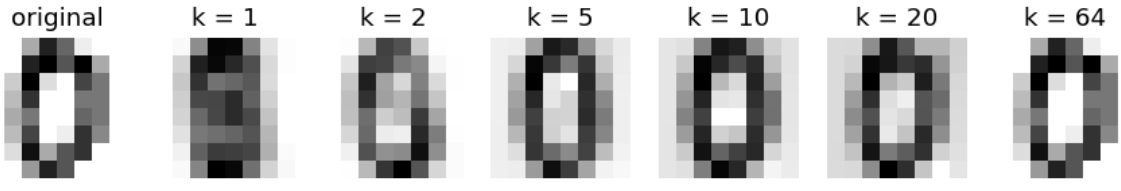
Out[2]: (1797, 64)

A simple experiment:

1. Describe each digit by its first k coordinates in a well-chosen basis (we will derive that basis, the principal components, in Section 2.4).
2. Rebuild the image from those k numbers alone.

In [3]:
```
pca_full = PCA().fit(X)
x0 = X[0:1]

ks = [1, 2, 5, 10, 20, 64]
fig, axes = plt.subplots(1, len(ks) + 1, figsize=(10, 1.8))
axes[0].imshow(x0.reshape(8, 8), cmap="gray_r")
axes[0].set_title("original")
for ax, k in zip(axes[1:], ks):
    p = PCA(n_components=k).fit(X)
    rec = p.inverse_transform(p.transform(x0))
    ax.imshow(rec.reshape(8, 8), cmap="gray_r")
    ax.set_title(f"k = {k}")
for ax in axes:
    ax.axis("off")
plt.show()
```

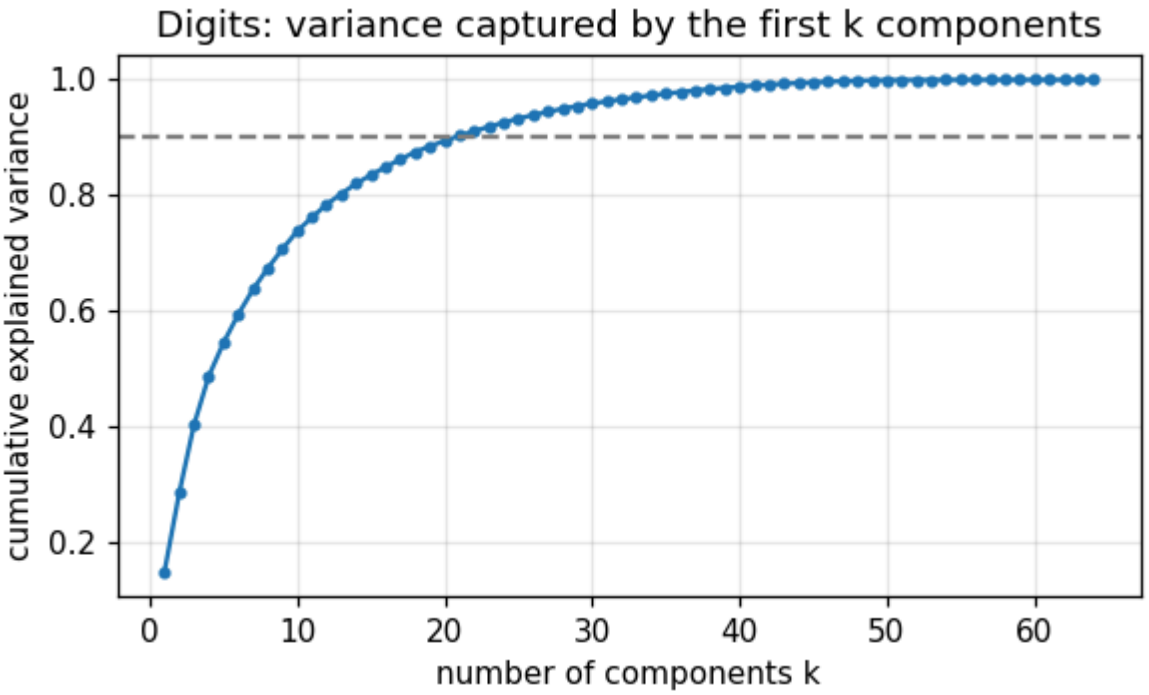


In [4]:
```
cum = np.cumsum(pca_full.explained_variance_ratio_)

plt.figure(figsize=(6, 3.2))
```

```
plt.plot(np.arange(1, 65), cum, marker=".")
plt.axhline(0.9, ls="--", c="gray")
plt.xlabel("number of components k")
plt.ylabel("cumulative explained variance")
plt.title("Digits: variance captured by the first k components")
plt.grid(alpha=0.3)
plt.show()

{"k for 90%": int(np.searchsorted(cum, 0.90) + 1),
 "k for 95%": int(np.searchsorted(cum, 0.95) + 1)}
```



Out[4]: `{'k for 90%': 21, 'k for 95%': 29}`

Good news:

- about **21 numbers out of 64** keep 90% of the variance
- 10 numbers already give a recognisable digit
- the data occupy a small part of $\mathbb{R}^{64}$

Bad news:

- 21 dimensions is far beyond our geometric intuition
- in case of a 224×224 colour image, we have 150 528 numbers...

2. Mathematical foundations

2.1 Inner products and similarity

An **inner product** on $\mathbb{R}^d$ is a map $\langle \cdot, \cdot \rangle : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ such that for all $x, y, z \in \mathbb{R}^d$ and $\alpha, \beta \in \mathbb{R}$:

$$
\begin{aligned}
&\text{(IP1) symmetry:} && \langle x, y \rangle = \langle y, x \rangle, \\
&\text{(IP2) linearity in the first argument:} && \langle \alpha x + \beta z,\, y \rangle = \alpha \langle x, y \rangle + \beta \langle z, y \rangle, \\
&\text{(IP3) positive definiteness:} && \langle x, x \rangle \geq 0, \quad \langle x, x \rangle = 0 \iff x = 0.
\end{aligned}
$$

By (IP1), linearity carries over to the second argument, $\langle x, \alpha y + \beta z\rangle = \alpha\langle x, y\rangle + \beta\langle x, z\rangle$, so an inner product is a *symmetric, positive definite bilinear form*.

Every inner product is a matrix. Expanding $x = \sum_i x_i e_i$ and $y = \sum_j y_j e_j$ in the standard basis and using bilinearity,

$$\langle x, y\rangle = \sum_{i,j} x_i\, y_j\, \langle e_i, e_j\rangle = x^\top A\, y, \qquad A_{ij} = \langle e_i, e_j\rangle.$$

(IP1) makes A symmetric and (IP3) makes it positive definite. Conversely, every symmetric positive definite A defines an inner product. The standard one has $A = I$, i.e. $\langle x, y\rangle = \sum_i x_i y_i$.

An inner product induces the norm $\|x\| = \sqrt{\langle x, x\rangle}$ and, through Cauchy–Schwarz $|\langle x, y\rangle| \le \|x\|\,\|y\|$, the angle

$$\cos\theta(x, y) = \frac{\langle x, y\rangle}{\|x\|\,\|y\|} \in [-1, 1].$$

Three similarity notions we use constantly.

measure	formula	invariant to
dot product	$\langle x, y\rangle$	nothing (rewards length)
cosine similarity	$\langle x, y\rangle / (\|x\| \|y\|)$	positive rescaling of either vector
Euclidean distance	$\|x - y\|_2$	rotations and translations

Cosine and Euclidean agree on the sphere. If $\|u\| = \|v\| = 1$ then

$$\|u - v\|_2^2 = \|u\|^2 + \|v\|^2 - 2\langle u, v\rangle = 2 - 2\cos\theta(u, v).$$

The map $c \mapsto \sqrt{2 - 2c}$ is strictly decreasing, so *after ℓ_2 normalisation, ranking by cosine similarity and ranking by Euclidean distance give exactly the same neighbours.* This is why vector databases often store normalised embeddings and use whichever is cheaper.

In [5]:
```
# Word counts for three tiny self-written "documents" over the vocabulary
# [data, vector, model, cat, dog]. Document c is document a written twice.

a = np.array([3.0, 2.0, 1.0, 0.0, 0.0]) #simple bag-of-words with the number of
b = np.array([0.0, 0.0, 1.0, 3.0, 2.0])
c = 2.0 * a

def cosine(u, v):
    return float(u @ v / (np.linalg.norm(u) * np.linalg.norm(v)))
```

```
{"euclid(a, b)": float(np.linalg.norm(a - b)), "euclid(a, c)": float(np.linalg.n
 "cos(a, b)": cosine(a, b), "cos(a, c)": cosine(a, c)}
```

Out[5]:
```
{'euclid(a, b)': 5.0990195135927845,
 'euclid(a, c)': 3.7416573867739413,
 'cos(a, b)': 0.07142857142857142,
 'cos(a, c)': 1.0}
```

- in raw Euclidean terms the doubled document c is almost as far from a as the unrelated document b
- cosine similarity correctly reports that a and c say the same thing
- length is a nuisance variable for text (cosine dominates there)

A quick numerical check of the ranking equivalence on random unit vectors:

In [6]:
```
V = rng.standard_normal((500, 32))             # 500 vectors in R^{32} with coor
V /= np.linalg.norm(V, axis=1, keepdims=True)  # vector normalization
q = V[0]                                       # first vector plays the role of

rank_by_cos = np.argsort(-(V @ q))             # V@q is the inner product (minus
rank_by_l2 = np.argsort(np.linalg.norm(V - q, axis=1))
bool(np.array_equal(rank_by_cos, rank_by_l2))
```

Out[6]:
```
True
```

Mahalanobis distance. Euclidean distance silently assumes that all features are measured in the same units, have the same spread and are uncorrelated: every direction counts equally. Real features rarely behave like that. Rescaling one coordinate (metres to millimetres) changes which points are "close", and two strongly correlated coordinates count the same information twice. The remedy is to let the data define the geometry through their covariance matrix Σ (symmetric, assumed positive definite):

$$d_M(x, y) = \sqrt{(x - y)^\top \Sigma^{-1} (x - y)}.$$

This is exactly the norm induced by the inner product $\langle x, y \rangle_{\Sigma^{-1}} = x^\top \Sigma^{-1} y$, i.e. the matrix form of Section 2.1 with $A = \Sigma^{-1}$. Since Σ^{-1} is symmetric positive definite, d_M is a genuine metric.

Special cases.

- $\Sigma = I$: ordinary Euclidean distance.
- $\Sigma = \mathrm{diag}(s_1^2, \ldots, s_d^2)$: the *standardised* Euclidean distance $\sqrt{\sum_i (x_i - y_i)^2 / s_i^2}$, i.e. every feature measured in units of its own standard deviation.
- General Σ: additionally removes correlations.

Spectral view. Diagonalise $\Sigma = V \Lambda V^\top$, with orthonormal eigenvectors v_i and eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d > 0$. Then $\Sigma^{-1} = V \Lambda^{-1} V^\top$ and

$$d_M^2(x, y) = \sum_{i=1}^{d} \frac{\left(v_i^\top (x - y)\right)^2}{\lambda_i}.$$

Displacements are measured along the principal axes of the data, and each component is divided by the variance in that direction. A step along a direction in which the data hardly vary (small λ_i) is heavily penalised; the same step along a high-variance direction is cheap.

Whitening. Put $W = \Sigma^{-1/2} = V\Lambda^{-1/2}V^\top$. Then $W^\top W = \Sigma^{-1}$, so

$$d_M(x, y) = \|Wx - Wy\|_2, \qquad \mathrm{Cov}(Wx) = W\Sigma W^\top = I.$$

Mahalanobis distance is Euclidean distance after a linear map that makes the data isotropic: rotate to the principal axes ($V^\top$), rescale each axis by $1/\sqrt{\lambda_i}$, and rotate back. Any W with $W^\top W = \Sigma^{-1}$ works, e.g. $\Lambda^{-1/2}V^\top$ (PCA whitening, Section 2.4) or the inverse Cholesky factor.

Invariance. For any invertible affine change of variables $x' = Bx + b$ the covariance becomes $B\Sigma B^\top$, and a direct substitution shows $d_M(x', y') = d_M(x, y)$. Mahalanobis distance does not depend on units, scaling or any linear mixing of the features.

Probabilistic meaning. For $X \sim \mathcal{N}(\mu, \Sigma)$ the density is $\propto \exp\!\left(-\tfrac{1}{2}d_M^2(x, \mu)\right)$, so the level sets of $d_M(\cdot, \mu)$ are exactly the density contours (ellipsoids). Moreover $W(X - \mu) \sim \mathcal{N}(0, I)$, hence

$$d_M^2(X, \mu) \sim \chi^2_d.$$

This turns the distance into a test: a point is an outlier at level α when $d_M^2(x, \mu) > \chi^2_{d,1-\alpha}$. For $d = 2$ the tail has the closed form $P\big(d_M^2 > t\big) = e^{-t/2}$.

In practice Σ is estimated.

- the sample covariance needs $n \gg d$
- for d close to or above n:
  - it is singular or badly conditioned
  - $\hat{\Sigma}^{-1}$ amplifies noise in exactly the small-variance directions that d_M weights most
  - this is the curse of dimensionality of Section 2.2 in another guise
  - standard fixes are shrinkage ($\hat{\Sigma} + \lambda I$, Ledoit–Wolf):
    - keeping only the top principal components
    - or keeping a diagonal Σ
- outliers inflate $\hat{\Sigma}$ and can hide themselves ("masking")
- robust estimators such as the Minimum Covariance Determinant (`sklearn.covariance.MinCovDet`) avoid this.

Where it appears. Multivariate outlier

In [7]:
```
Sigma = np.array([[1.0, 0.9], [0.9, 1.0]])
pts = rng.multivariate_normal([0.0, 0.0], Sigma, size=1500)
Sinv = np.linalg.inv(Sigma)

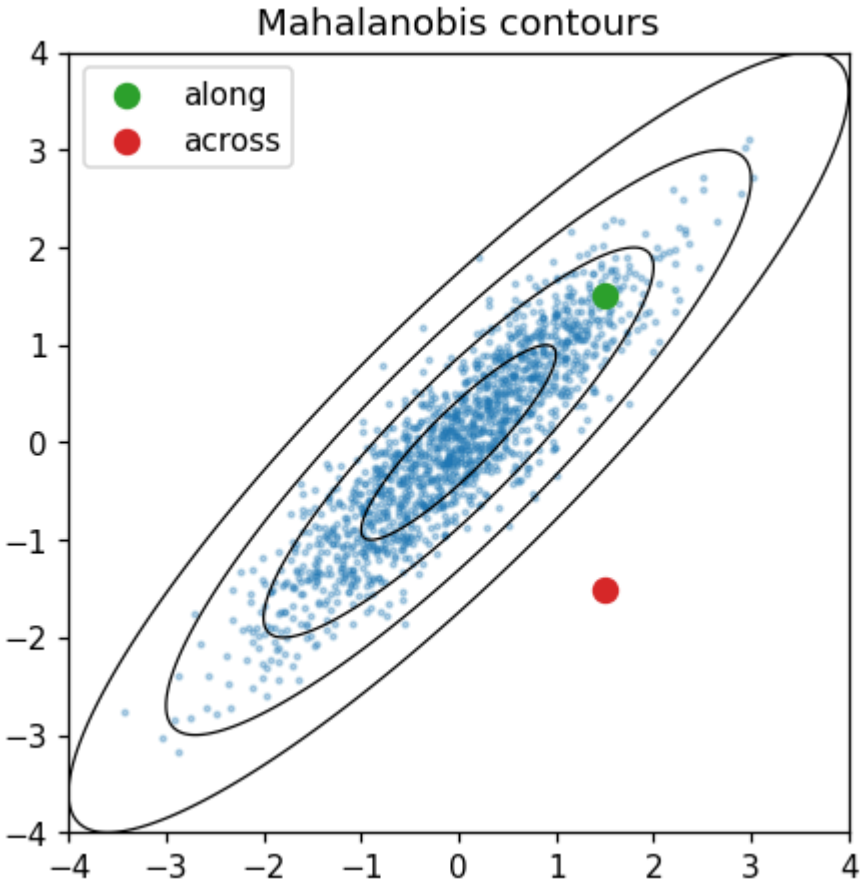
p_along = np.array([1.5, 1.5])       # along the main axis of the cloud
p_across = np.array([1.5, -1.5])     # across it
```

```
def mahalanobis(x, Sinv):
    return float(np.sqrt(x @ Sinv @ x))

g = np.linspace(-4, 4, 200)
G1, G2 = np.meshgrid(g, g)
G = np.stack([G1.ravel(), G2.ravel()], axis=1)
M = np.sqrt(np.einsum("ij,jk,ik->i", G, Sinv, G)).reshape(G1.shape)

plt.figure(figsize=(4.6, 4.6))
plt.scatter(pts[:, 0], pts[:, 1], s=3, alpha=0.3)
plt.contour(G1, G2, M, levels=[1, 2, 3, 4], colors="k", linewidths=0.8)
plt.scatter(*p_along, c="tab:green", s=60, label="along")
plt.scatter(*p_across, c="tab:red", s=60, label="across")
plt.gca().set_aspect("equal")
plt.legend()
plt.title("Mahalanobis contours")
plt.show()

{"euclid along": float(np.linalg.norm(p_along)), "euclid across": float(np.linal
 "mahalanobis along": mahalanobis(p_along, Sinv), "mahalanobis across": mahalano
```



Out[7]:
```
{'euclid along': 2.1213203435596424,
 'euclid across': 2.1213203435596424,
 'mahalanobis along': 1.5389675281277313,
 'mahalanobis across': 6.70820393249937}
```

- both points are at the same Euclidean distance ≈ 2.12 from the mean
- their Mahalanobis distances differ by a factor of more than four
- the spectral formula explains why:

- the covariance $\Sigma = \begin{pmatrix} 1 & 0.9 \\ 0.9 & 1 \end{pmatrix}$ has eigenvalues $\lambda_1 = 1.9$ along $v_1 = (1,1)/\sqrt{2}$ and $\lambda_2 = 0.1$ across it, along $v_2 = (1,-1)/\sqrt{2}$
- each point has projection length $1.5\sqrt{2}$ on its own axis, so

$$d_M^2(\text{green}) = \frac{4.5}{1.9} \approx 2.37, \qquad d_M^2(\text{red}) = \frac{4.5}{0.1} = 45.$$

- under the fitted Gaussian ($d = 2$, so $P(d_M^2 > t) = e^{-t/2}$), a point at least as extreme as the green one occurs with probability ≈ 0.31, which is entirely ordinary
- for the red one the probability is $\approx 1.7 \cdot 10^{-10}$, which makes it an outlier
- Euclidean distance cannot see the difference, because it ignores the shape of the cloud

2.2 The curse of dimensionality

> Richard Bellman (1961)

To tabulate a function of d variables on a grid with spacing h in the unit cube, one needs $(1/h)^d$ grid points:

- ten points per axis means 10^d cells,
- already 10^{10} for $d = 10$.

The same exponential growth hits anyone who wants to *cover* a high-dimensional space with data.

Today the term covers several related phenomena.

Local neighbourhoods are not local. Let data be uniform in $[0,1]^d$, and suppose we want a cubical neighbourhood that contains a fraction p of the points (a "local" average, a k-NN ball). Its volume must be p (because for the uniform distribution on $[0,1]^d$ *probability equals volume*), so its edge length is

$$e_d(p) = p^{1/d}.$$

For $d = 10$:

- capturing 1% of the data needs $e_{10}(0.01) \approx 0.63$,
- 10% needs ≈ 0.80, i.e. most of the range of every coordinate (Hastie et al., §2.5)

In high dimension a neighbourhood large enough to contain any data is no longer local, and methods that rely on local averaging (k-NN, kernel smoothing) lose their footing.

Volume escapes to the corners and to the surface. The volume of the Euclidean ball of radius r in $\mathbb{R}^d$ is

$$V_d(r) = \frac{\pi^{d/2}}{\Gamma\left(\frac{d}{2}+1\right)} r^d,$$

with the familiar special cases

$$V_1(1) = 2,\ V_2(1) = \pi,\ V_3(1) = \tfrac{4}{3}\pi.$$

A convenient recursion follows from $\Gamma(z+1) = z\Gamma(z)$:

$$V_d(1) = \frac{2\pi}{d}\, V_{d-2}(1).$$

Since $2\pi/d > 1$ exactly for $d \le 6$:

- the even-dimensional volumes grow up to $d = 6$
- the odd-dimensional ones grow up to $d = 5$
- $V_5(1) = 8\pi^2/15 \approx 5.264 > V_6(1) = \pi^3/6 \approx 5.168$
- hence $V_d(1)$ decreases for $d > 5$.

Stirling's formula gives the rate:

$$V_d(1) \sim \frac{1}{\sqrt{\pi d}} \left(\frac{2\pi e}{d}\right)^{d/2} \longrightarrow\ 0.$$

Corners. The cube $[-1,1]^d$ has volume 2^d, and its inscribed ball (radius 1, touching the centre of every face) occupies the fraction $V_d(1)/2^d$:

- $\pi/4 \approx 0.79$ in the plane,
- $\pi/6 \approx 0.52$ in space,
- ≈ 0.0025 for $d = 10$,
- $\approx 2.5 \cdot 10^{-8}$ for $d = 20$.

Since the uniform density on the cube is $1/2^d$, this fraction is also the probability that a uniformly drawn point falls into the ball: such points almost never lie near the centre.

Where does the rest of the volume go?

- the cube has 2^d vertices $(\pm 1, \ldots, \pm 1)$
- the centre is at distance 1 from every face
- by Pythagoras, the center is at distance $\sqrt{d}$ from every vertex
  - for $d = 100$ the vertices are ten times farther away than the faces
  - a high-dimensional cube is best pictured as a small core with a huge number of long spikes pointing towards the vertices
  - almost all of its volume sits in the spikes

Surface. Within the ball, the fraction of volume in the outer shell of relative thickness ε is

$$\frac{V_d(1) - V_d(1-\varepsilon)}{V_d(1)} = 1 - (1-\varepsilon)^d \ \approx\ 1 - e^{-\varepsilon d} \xrightarrow[d\to\infty]{}\ 1.$$

For $\varepsilon = 0.05$ this is:

- 0.40 in $d = 10$,
- 0.92 in $d = 50$,
- 0.994 in $d = 100$.

Equivalently, half of the volume lies within relative distance $1 - 2^{-1/d} \approx (\ln 2)/d$ of the surface:

- under 1% of the radius for $d = 100$,
- a uniformly drawn point is almost surely near the boundary.

The Gaussian version. The same happens without any boundary. For $x \sim \mathcal{N}(0, I_d)$, $\|x\|^2 \sim \chi^2_d$ has mean d and variance $2d$, so

$$\|x\| \approx \sqrt{d} \pm \tfrac{1}{\sqrt{2}}.$$

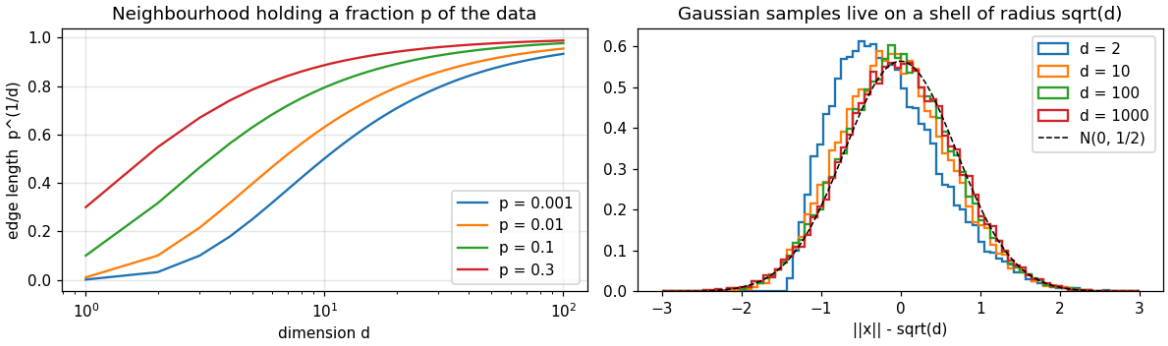
The density is largest at the origin, yet almost all of the probability mass sits in a thin shell of radius $\sqrt{d}$ and constant width. A high-dimensional Gaussian looks less like a ball than a soap bubble. This matters later: the latent spaces of generative models (Week 14) are Gaussian, and "typical" samples live on that shell, not near the mean.

In [8]:
```
d_grid = np.arange(1, 101)
fig, axes = plt.subplots(1, 2, figsize=(11, 3.4))

# Left: edge length of a subcube holding a fraction p of uniform data in [0, 1]^
for p in [0.001, 0.01, 0.1, 0.3]:
    axes[0].plot(d_grid, p ** (1.0 / d_grid), label=f"p = {p}")
axes[0].set_xscale("log")
axes[0].set_xlabel("dimension d")
axes[0].set_ylabel("edge length  p^(1/d)")
axes[0].set_title("Neighbourhood holding a fraction p of the data")
axes[0].legend()
axes[0].grid(alpha=0.3)

# Right: norms of standard Gaussian samples, centred at sqrt(d).
grid = np.linspace(-3, 3, 400)
norm_stats = {}
for dd in [2, 10, 100, 1000]:
    r = np.linalg.norm(rng.standard_normal((20_000, dd)), axis=1)
    norm_stats[dd] = {"mean / sqrt(d)": round(float(r.mean() / np.sqrt(dd)), 3),
                      "sd": round(float(r.std()), 3)}
    axes[1].hist(r - np.sqrt(dd), bins=80, range=(-3, 3), density=True,
                 histtype="step", lw=1.5, label=f"d = {dd}")
axes[1].plot(grid, np.exp(-grid**2) / np.sqrt(np.pi), c="k", ls="--", lw=1, labe
axes[1].set_xlabel("||x|| - sqrt(d)")
axes[1].set_title("Gaussian samples live on a shell of radius sqrt(d)")
axes[1].legend()
plt.tight_layout()
plt.show()

{"edge for 1% of data, d=10": round(0.01 ** 0.1, 3),
 "edge for 10% of data, d=10": round(0.1 ** 0.1, 3),
 "Gaussian norms": norm_stats}
```

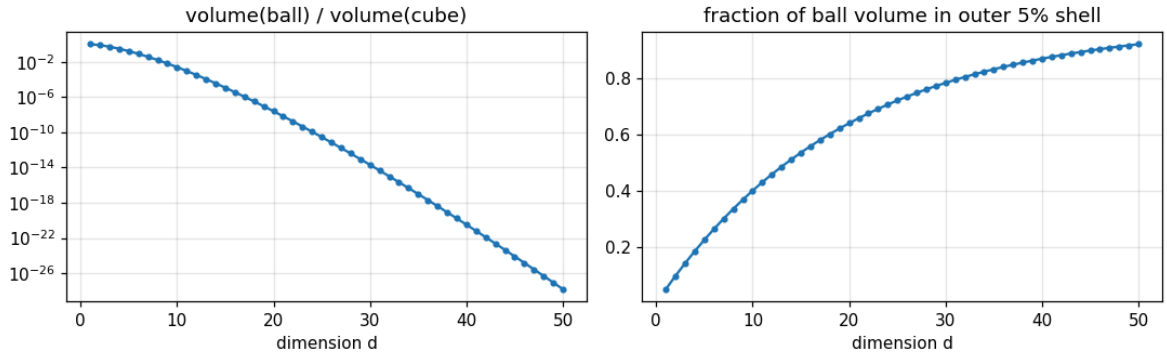


Out[8]:
```
{'edge for 1% of data, d=10': 0.631,
 'edge for 10% of data, d=10': 0.794,
 'Gaussian norms': {2: {'mean / sqrt(d)': 0.887, 'sd': 0.66},
  10: {'mean / sqrt(d)': 0.977, 'sd': 0.694},
  100: {'mean / sqrt(d)': 0.997, 'sd': 0.707},
  1000: {'mean / sqrt(d)': 1.0, 'sd': 0.712}}}
```

In [9]:
```
d = np.arange(1, 51)
log_ball = (d / 2) * np.log(np.pi) - gammaln(d / 2 + 1)
ball_over_cube = np.exp(log_ball - d * np.log(2.0))
shell = 1.0 - (1.0 - 0.05) ** d

fig, axes = plt.subplots(1, 2, figsize=(10, 3.2))
axes[0].semilogy(d, ball_over_cube, marker=".")
axes[0].set_xlabel("dimension d")
axes[0].set_title("volume(ball) / volume(cube)")
axes[0].grid(alpha=0.3)
axes[1].plot(d, shell, marker=".")
axes[1].set_xlabel("dimension d")
axes[1].set_title("fraction of ball volume in outer 5% shell")
axes[1].grid(alpha=0.3)
plt.tight_layout()
plt.show()

{"d=10": float(ball_over_cube[9]), "d=20": float(ball_over_cube[19])}
```



Out[9]:
```
{'d=10': 0.00249039457019272, 'd=20': 2.461136950494197e-08}
```

(b) Distances concentrate. Let x, y be independent with i.i.d. coordinates, and $Z_i = (x_i - y_i)^2$ with mean μ and variance σ^2. Then $\|x-y\|^2 = \sum_{i=1}^{d} Z_i$ has mean $d\mu$ and standard deviation $\sigma\sqrt{d}$, so

$$\frac{\mathrm{sd}\|x-y\|^2}{\mathbb{E}\|x-y\|^2} = \frac{\sigma}{\mu\sqrt{d}} \longrightarrow 0.$$

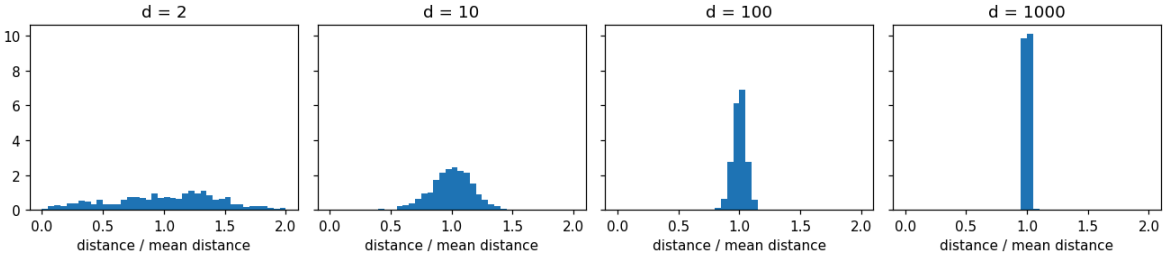
For Gaussian data the statement is exact: $x - y \sim \mathcal{N}(0, 2I_d)$, so $\|x-y\|^2/2 \sim \chi^2_d$. More generally, Beyer et al. (1999) showed that under mild conditions the **relative contrast** between the farthest and the nearest neighbour of a query,

$$\frac{D_{\max} - D_{\min}}{D_{\min}} \xrightarrow{P} 0 \quad (d \to \infty),$$

so "nearest" loses its meaning for unstructured noise. The histograms below rescale all distances by their mean: as d grows, they collapse onto 1.

In [10]:
```
dims = [2, 10, 100, 1000]
fig, axes = plt.subplots(1, len(dims), figsize=(12, 2.8), sharey=True)
contrast = {}
for ax, dd in zip(axes, dims):
    Z = rng.random((1000, dd))
    query = rng.random(dd)
    dist = np.linalg.norm(Z - query, axis=1)
    contrast[dd] = float((dist.max() - dist.min()) / dist.min())
    ax.hist(dist / dist.mean(), bins=40, range=(0, 2), density=True)
    ax.set_title(f"d = {dd}")
    ax.set_xlabel("distance / mean distance")
plt.tight_layout()
plt.show()

contrast
```



Out[10]:
```
{2: 83.040448007016,
 10: 2.6273970570331318,
 100: 0.5413575535057675,
 1000: 0.10845616795168615}
```

(c) Random directions are almost orthogonal.

Setting and goal. Draw two independent random vectors $u, v \sim \mathcal{N}(0, I_d)$: every coordinate is an independent standard normal. Their directions are uniformly distributed on the sphere, so they model "two arbitrary directions with nothing in common". We want to know what angle they typically make, i.e. how the cosine

$$\cos\theta(u,v) = \frac{\langle u, v\rangle}{\|u\|\,\|v\|}$$

behaves as d grows.

The inner product is a sum of d independent terms, $\langle u, v\rangle = \sum_{i=1}^{d} u_i v_i$. Each term has

$$\mathbb{E}[u_i v_i] = \mathbb{E}[u_i]\,\mathbb{E}[v_i] = 0, \qquad \mathrm{Var}(u_i v_i) = \mathbb{E}[u_i^2]\,\mathbb{E}[v_i^2] = 1.$$

The terms are equally likely to be positive or negative, so they largely cancel. For independent zero-mean terms the *variances* add, so the sum $S = \langle u, v \rangle$ has mean 0 and variance d; its typical magnitude is the standard deviation $\sqrt{\mathbb{E}[S^2]} = \sqrt{d}$ (and $S \approx \mathcal{N}(0, d)$ by the CLT).

The squared norm $\|u\|^2 = \sum_i u_i^2 \sim \chi^2_d$ is a sum of *positive* terms, so nothing cancels: it has mean d and standard deviation $\sqrt{2d}$, hence $\|u\| \approx \sqrt{d}$ with vanishing relative error. The same holds for v, so $\|u\|\,\|v\| \approx d$.

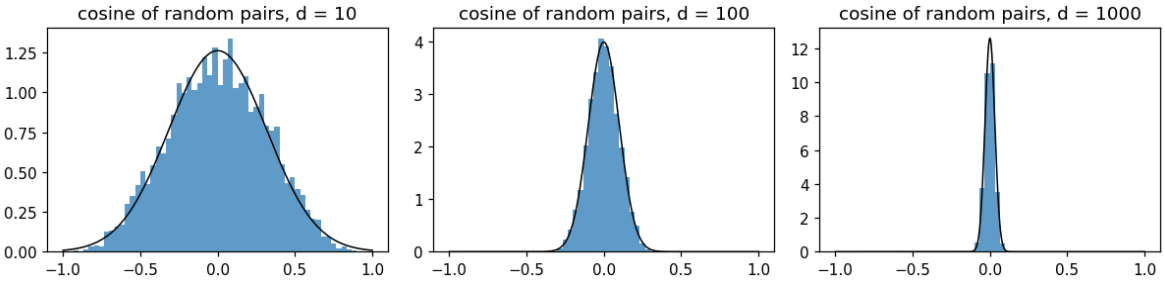
A numerator of size $\sqrt{d}$ over a denominator of size d gives

$$\cos\theta(u, v) \approx \frac{\mathcal{N}(0, d)}{d} = \mathcal{N}\left(0, \tfrac{1}{d}\right).$$

The cosine shrinks like $1/\sqrt{d}$. The whole effect is the contrast between *cancellation* in the numerator (random signs, growth like $\sqrt{d}$) and *accumulation* in the denominator (squares, growth like d).

In $\mathbb{R}^{1000}$ two random vectors make an angle of $90^\circ \pm 2^\circ$. This is a blessing as well as a curse: a high-dimensional space has room for exponentially many *nearly* orthogonal directions, which is part of why embedding spaces of a few hundred dimensions can represent so many distinct concepts.

In [11]:
```
fig, axes = plt.subplots(1, 3, figsize=(11, 2.8), sharey=False)
for ax, dd in zip(axes, [10, 100, 1000]):
    U = rng.standard_normal((5000, dd))
    W = rng.standard_normal((5000, dd))
    cs = np.sum(U * W, axis=1) / (np.linalg.norm(U, axis=1) * np.linalg.norm(W,
    grid = np.linspace(-1, 1, 400)
    ax.hist(cs, bins=60, range=(-1, 1), density=True, alpha=0.7)
    ax.plot(grid, np.sqrt(dd / (2 * np.pi)) * np.exp(-dd * grid**2 / 2), c="k",
    ax.set_title(f"cosine of random pairs, d = {dd}")
plt.tight_layout()
plt.show()
```



What it costs in practice. Take the digits and append m coordinates of pure uniform noise on the same scale as the pixels. The informative signal is untouched, yet a 5-nearest-neighbour classifier degrades steadily, because the noise coordinates dominate every distance.

In [12]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

Xs = X / 16.0
knn_acc = {}
```

```
for m in [0, 64, 256, 1024]:
    Xn = np.hstack([Xs, rng.random((len(Xs), m))])
    knn_acc[m] = round(float(cross_val_score(KNeighborsClassifier(5), Xn, y, cv=
knn_acc
```

Out[12]: {0: 0.963, 64: 0.942, 256: 0.866, 1024: 0.608}

The way out: intrinsic dimension. The theorems above assume coordinates that are independent and equally informative. Real unstructured data are nothing like that: pixels of a digit are strongly correlated and the set of plausible digits is close to a low-dimensional manifold (Part 1). Learning a good representation largely means *finding the few directions that matter* — which is exactly what dimensionality reduction does.

2.3 Random projections and the Johnson–Lindenstrauss lemma

The cheapest dimensionality reduction ignores the data entirely: multiply every vector by the same random matrix. Surprisingly, this preserves all pairwise distances almost exactly.

Lemma (Johnson–Lindenstrauss, 1984; constants of Dasgupta & Gupta, 2003). Let $0 < \varepsilon < 1$ and let $x_1, \dots, x_n \in \mathbb{R}^d$ be arbitrary points. If

$$k \;\geq\; \frac{4 \ln n}{\varepsilon^2/2 - \varepsilon^3/3},$$

then there exists a linear map $f : \mathbb{R}^d \to \mathbb{R}^k$ such that for **all** pairs i, j

$$(1-\varepsilon)\,\|x_i - x_j\|^2 \;\leq\; \|f(x_i) - f(x_j)\|^2 \;\leq\; (1+\varepsilon)\,\|x_i - x_j\|^2.$$

Moreover, a random map works: take $f(x) = R^\top x$ with $R \in \mathbb{R}^{d \times k}$ and independent entries $R_{ij} \sim \mathcal{N}(0, 1/k)$.

Proof sketch

Since f is linear, $f(x_i) - f(x_j) = f(x_i - x_j)$. It is therefore enough to show that f approximately preserves the norm of each of the difference vectors $u = x_i - x_j$.

Fix $u \neq 0$. Each coordinate of $R^\top u$ is a linear combination of independent Gaussians,

$$(R^\top u)_l = \sum_{m=1}^{d} R_{ml}\, u_m \sim \mathcal{N}\!\left(0,\; \frac{\|u\|^2}{k}\right), \qquad l = 1, \dots, k,$$

and the k coordinates are independent. Dividing each by its standard deviation $\|u\|/\sqrt{k}$ gives standard normals, so

$$\frac{k\,\|R^\top u\|^2}{\|u\|^2} \sim \chi^2_k, \qquad \text{hence} \qquad \mathbb{E}\,\frac{\|R^\top u\|^2}{\|u\|^2} = 1.$$

The projection preserves squared lengths on average; the question is how much it fluctuates.

A χ^2_k/k variable concentrates around 1 (Chernoff bound applied to the sum of k squared Gaussians):

$$P\left(\left|\frac{\|R^\top u\|^2}{\|u\|^2} - 1\right| > \varepsilon\right) \;\le\; 2\exp\left(-\frac{k}{2}\left(\frac{\varepsilon^2}{2} - \frac{\varepsilon^3}{3}\right)\right).$$

There are $\binom{n}{2} < n^2/2$ pairs. With k as in the lemma, the exponential is at most $e^{-2\ln n} = 1/n^2$, so

$$P(\text{some pair fails}) \;\le\; \frac{n^2}{2}\cdot\frac{2}{n^2}\cdot\frac{n-1}{n} \;<\; 1.$$

Precisely: $\binom{n}{2}\cdot 2/n^2 = (n-1)/n = 1 - 1/n$. A random R therefore succeeds with probability at least $1/n$, so a good map exists; in practice it succeeds far more often, and one can simply redraw R and check.

Takeaways

- k grows only logarithmically with the number of points n (doubling the data set adds a constant to k)
- k does not depend on the original dimension d at all (projecting from 10^4 or 10^6 dimensions needs the same k)
- The price is $1/\varepsilon^2$ (high accuracy is expensive: halving ε quadruples k)
- The map is data-oblivious. R is drawn without looking at the data, so it is cheap, streaming-friendly and can be applied to points that arrive later.
- Typical uses: very large d with moderate distortion acceptable, such as sketching, fast approximate nearest-neighbour search, and preprocessing before a heavier method.
- The mechanism is the same concentration of χ^2 variables that we met in Section 2.2.

In [13]:
```
from sklearn.random_projection import johnson_lindenstrauss_min_dim

{eps: int(johnson_lindenstrauss_min_dim(n_samples=2000, eps=eps)) for eps in [0.
```

Out[13]:
```
{0.1: 6515, 0.2: 1754, 0.3: 844, 0.5: 364}
```

The bound is pessimistic; in practice much smaller k already works well. We check empirically on 2000 points in $\mathbb{R}^{10\,000}$, measuring the ratio projected / original distance over all $\approx 2\cdot 10^6$ pairs.

In [14]:
```
import torch

def get_device():
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")

def synchronize(device):
    if device.type == "cuda":
        torch.cuda.synchronize()
```

```
    elif device.type == "mps":
        torch.mps.synchronize()

def pairwise_distances(T):
    sq = (T * T).sum(dim=1, keepdim=True)
    return (sq + sq.T - 2.0 * (T @ T.T)).clamp_min(0.0).sqrt()

device = get_device()
device
```

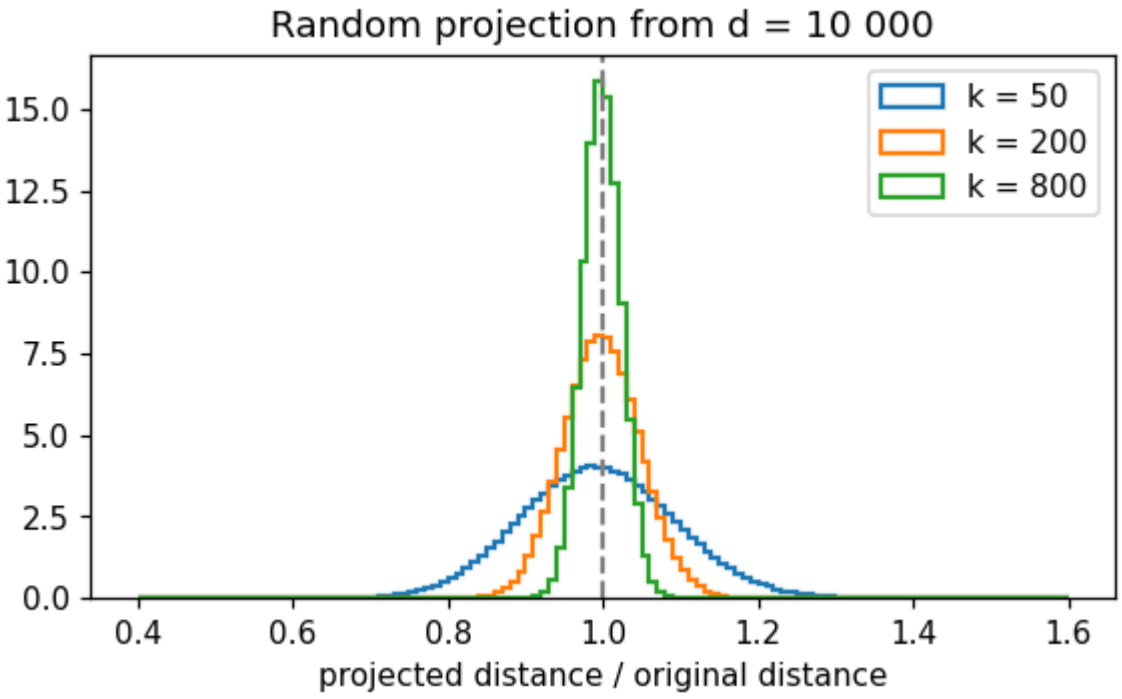
Out[14]: device(type='mps')

In [15]:
```
def jl_distortion(X, k, device, seed=0):
    # Ratio ||R^T x_i - R^T x_j|| / ||x_i - x_j|| over all pairs i < j.
    gen = torch.Generator().manual_seed(seed)
    A = torch.as_tensor(X, dtype=torch.float32, device=device)
    R = (torch.randn(A.shape[1], k, generator=gen) / k**0.5).to(device)
    D0 = pairwise_distances(A)
    D1 = pairwise_distances(A @ R)
    iu = torch.triu_indices(A.shape[0], A.shape[0], offset=1, device=device)
    return (D1[iu[0], iu[1]] / D0[iu[0], iu[1]]).cpu().numpy()

X_jl = rng.standard_normal((2000, 10_000)).astype(np.float32)
ratios = {k: jl_distortion(X_jl, k, device) for k in [50, 200, 800]}

plt.figure(figsize=(6, 3.2))
for k, r in ratios.items():
    plt.hist(r, bins=120, range=(0.4, 1.6), density=True, histtype="step", lw=1.
plt.axvline(1.0, c="gray", ls="--")
plt.xlabel("projected distance / original distance")
plt.legend()
plt.title("Random projection from d = 10 000")
plt.show()

{k: {"min": round(float(r.min()), 3), "max": round(float(r.max()), 3), "sd": rou
 for k, r in ratios.items()}
```



```
Out[15]: {50: {'min': 0.548, 'max': 1.497, 'sd': 0.0986},
          200: {'min': 0.778, 'max': 1.255, 'sd': 0.0494},
          800: {'min': 0.881, 'max': 1.128, 'sd': 0.0248}}
```

- The spread shrinks like $1/\sqrt{2k}$, exactly as the χ^2_k argument predicts (for $k = 200$: $1/\sqrt{400} = 0.05$).
- Random projections preserve *all* distances on average but favour none.
- To keep the *important* directions we need to look at the data — that is PCA.

2.4 Principal component analysis

Setting. Data $x_1, \dots, x_n \in \mathbb{R}^d$ are stacked as rows of $X \in \mathbb{R}^{n \times d}$ and centred, $\sum_i x_i = 0$. The sample covariance is $C = \frac{1}{n-1} X^\top X$, symmetric and positive semidefinite.

View 1 — maximise the variance of a projection. The projection onto a unit vector w has sample variance

$$\frac{1}{n-1} \sum_i (w^\top x_i)^2 = w^\top C\, w.$$

Maximise $w^\top C w$ subject to $w^\top w = 1$. With the Lagrangian $\mathcal{L}(w, \lambda) = w^\top C w - \lambda(w^\top w - 1)$, setting $\nabla_w \mathcal{L} = 0$ gives

$$C\, w = \lambda\, w, \qquad w^\top C w = \lambda.$$

The optimum is the eigenvector of the largest eigenvalue, and the variance it captures *is* that eigenvalue. Requiring the next direction to be orthogonal to the first and repeating the argument gives the eigenvectors $v_1, \dots, v_d$ in order of $\lambda_1 \geq \dots \geq \lambda_d \geq 0$ (spectral theorem). These are the **principal axes**; the coordinates $z_i = V_k^\top x_i$ are the **scores**.

View 2 — minimise reconstruction error. Project onto a k-dimensional subspace with orthonormal basis $W \in \mathbb{R}^{d \times k}$. By Pythagoras,

$$\|x - W W^\top x\|^2 = \|x\|^2 - \|W^\top x\|^2,$$

so minimising the average reconstruction error is the same as maximising $\sum_i \|W^\top x_i\|^2 = (n-1)\,\mathrm{tr}(W^\top C W)$, which again is solved by the top k eigenvectors. Hence:

> Maximum variance and minimum reconstruction error are the same problem.

View 3 — the SVD of the data matrix. Write the thin SVD $X = U \Sigma V^\top$ with $\Sigma = \mathrm{diag}(\sigma_1 \geq \sigma_2 \geq \dots)$. Then

$$C = \frac{1}{n-1} V \Sigma U^\top U \Sigma V^\top = V \frac{\Sigma^2}{n-1} V^\top,$$

so the right singular vectors are the principal axes, $\lambda_i = \sigma_i^2/(n-1)$, and the scores are $XV = U\Sigma$. The **explained variance ratio** of component i is $\sigma_i^2 / \sum_j \sigma_j^2$.

Why compute PCA by the SVD of X and not by eigendecomposing C? Forming $X^\top X$ squares the condition number, $\kappa(C) = \kappa(X)^2$, so small singular values are lost to rounding; the SVD never forms C.

Theorem (Eckart–Young–Mirsky). The best rank-k approximation of X in Frobenius (and spectral) norm is the truncated SVD $X_k = U_k \Sigma_k V_k^\top$, with

$$\|X - X_k\|_F^2 = \sum_{i>k} \sigma_i^2.$$

PCA with k components is therefore the optimal linear compression of the centred data.

In [16]:
```
def pca_svd(X, k):
    mean = X.mean(axis=0)
    Xc = X - mean
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    scores = U[:, :k] * S[:k]                       # = Xc @ Vt[:k].T
    evr = S**2 / np.sum(S**2)
    return {"mean": mean, "components": Vt[:k], "scores": scores,
            "singular_values": S, "explained_variance_ratio": evr[:k]}

ours = pca_svd(X, 10)
ref = PCA(n_components=10).fit(X)

# Each component is defined only up to sign, so compare absolute values.
{"evr matches sklearn": bool(np.allclose(ours["explained_variance_ratio"], ref.e
 "scores match up to sign": bool(np.allclose(np.abs(ours["scores"]), np.abs(ref.
```

Out[16]:
```
{'evr matches sklearn': True, 'scores match up to sign': True}
```

In [17]:
```
# Eckart-Young: squared Frobenius error of the rank-k truncation equals the tail
Xc = X - X.mean(axis=0)
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
k = 10
Xk = (U[:, :k] * S[:k]) @ Vt[:k]
{"||Xc - Xk||_F^2": float(np.linalg.norm(Xc - Xk) ** 2), "sum_{i>k} sigma_i^2":
```

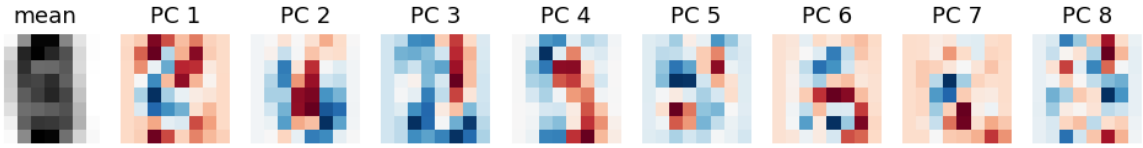
Out[17]:
```
{'||Xc - Xk||_F^2': 565183.4033224074,
 'sum_{i>k} sigma_i^2': 565183.4033224069}
```

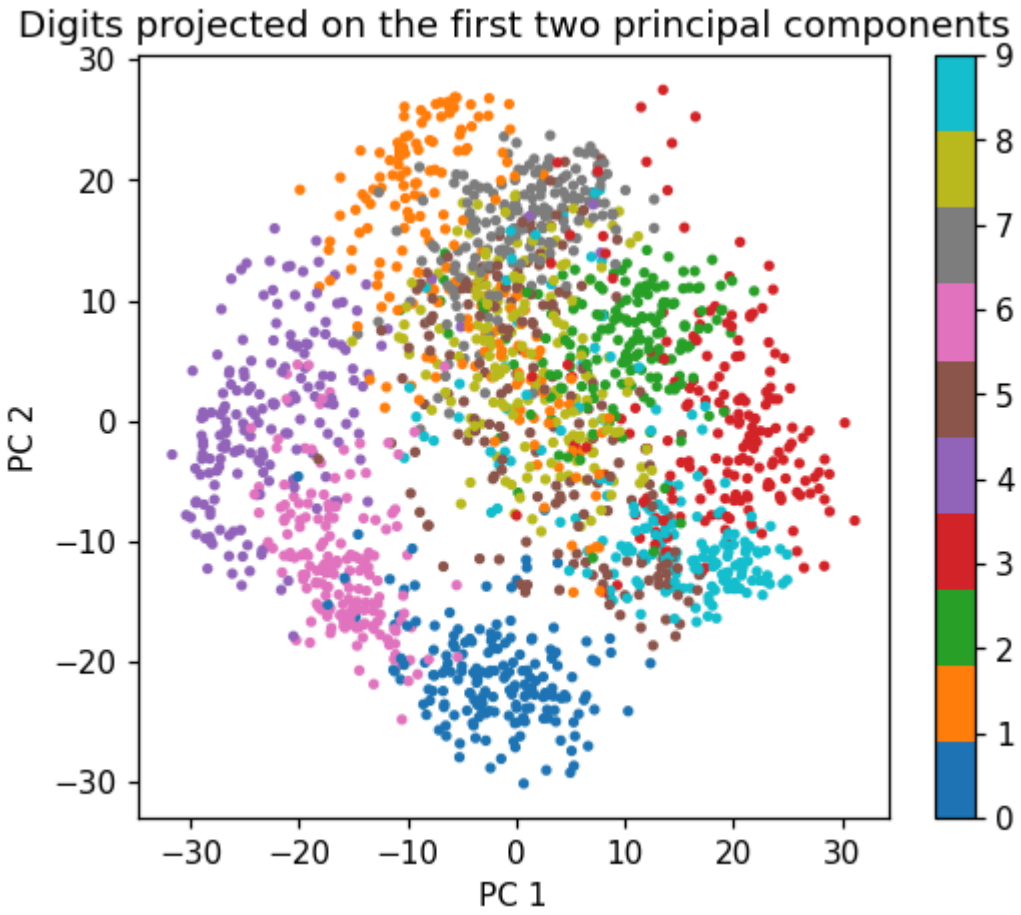
The principal axes live in pixel space, so for images we can look at them. The first ones are the dominant modes of variation among handwritten digits (sometimes called *eigendigits*).

In [18]:
```
fig, axes = plt.subplots(1, 9, figsize=(11, 1.6))
axes[0].imshow(ours["mean"].reshape(8, 8), cmap="gray_r")
axes[0].set_title("mean")
for i, ax in enumerate(axes[1:]):
    ax.imshow(ours["components"][i].reshape(8, 8), cmap="RdBu_r")
    ax.set_title(f"PC {i + 1}")
for ax in axes:
    ax.axis("off")
plt.show()

Z2 = ours["scores"][:, :2]
plt.figure(figsize=(5.5, 4.5))
```

```
sc = plt.scatter(Z2[:, 0], Z2[:, 1], c=y, cmap="tab10", s=6)
plt.colorbar(sc, ticks=range(10))
plt.xlabel("PC 1")
plt.ylabel("PC 2")
plt.title("Digits projected on the first two principal components")
plt.show()
```





Two components already separate 0, 4, 6 and part of 3; other classes overlap. With only 28.5% of the variance in the plane this is expected — PCA is a *global, linear* method and its two-dimensional picture is a compromise over all directions at once.

Practical remarks.

- *Always centre.* Without centring the first "component" mostly points at the mean.
- *Scale when units differ.* PCA is not scale-invariant; standardise mixed features (PCA on the correlation matrix). For pixels or word counts on a common scale, leave them.
- *Sign is arbitrary.* v and $-v$ are both valid; never interpret the sign alone.
- *Choosing k.* Explained-variance threshold, the elbow of the scree plot, or simply the downstream validation score.

2.5 Randomised PCA at scale

A full SVD of an $n \times d$ matrix costs $O(nd \min(n, d))$. When we need only $k \ll \min(n, d)$ components, the **randomised range finder** of Halko, Martinsson & Tropp (2011) is much cheaper and is built almost entirely from matrix products — exactly what GPUs do well.

1. Draw a Gaussian test matrix $\Omega \in \mathbb{R}^{d \times (k+p)}$ with a small oversampling p (say 10) and form $Y = X\Omega$. With high probability the columns of Y span (almost) the top-k left singular subspace of X.
2. **Power iterations.** Replace X by $(XX^\top)^q X$, whose singular values are σ_i^{2q+1}: the gap between the wanted and the unwanted part of the spectrum widens geometrically. Re-orthonormalise between products to avoid losing the small directions.
3. Orthonormalise Y to get Q, form the small matrix $B = Q^\top X \in \mathbb{R}^{(k+p) \times d}$, and take its exact SVD $B = \tilde{U}\Sigma V^\top$. Then $X \approx (Q\tilde{U})\Sigma V^\top$.

Orthonormalisation with matrix products only. QR is not available on every accelerator, so we use Cholesky-QR in its eigenvalue form: for the small Gram matrix $G = Y^\top Y = W\Lambda W^\top$, the matrix $Q = YW\Lambda^{-1/2}$ has $Q^\top Q = I$. Repeating it once (CholeskyQR2) restores orthogonality to working precision in `float32`. The only factorisations are of $(k+p) \times (k+p)$ matrices, done on the CPU in `float64`; everything large stays on the device. This also sidesteps the fact that some `torch.linalg` routines are not implemented on MPS.

In [19]:
```
def orthonormalize(Y):
    G = (Y.T @ Y).cpu().double()                    # small (l x l) Gram matrix
    lam, W = torch.linalg.eigh(G)
    lam = lam.clamp_min(lam.max() * 1e-12)
    T = (W / lam.sqrt()).float().to(Y.device)
    return Y @ T

def cholqr2(Y):
    return orthonormalize(orthonormalize(Y))

def randomized_pca(X, k, oversample=10, n_iter=4, device=None, seed=0):
    device = get_device() if device is None else device
    A = torch.as_tensor(X, dtype=torch.float32, device=device)
    mean = A.mean(dim=0, keepdim=True)
    A = A - mean
    gen = torch.Generator().manual_seed(seed)
    Omega = torch.randn(A.shape[1], k + oversample, generator=gen).to(device)
    Q = cholqr2(A @ Omega)
    for _ in range(n_iter):
        Q = cholqr2(A.T @ Q)
        Q = cholqr2(A @ Q)
    B = (Q.T @ A).cpu().double()                    # small (l x d) matrix
    _, S, Vt = torch.linalg.svd(B, full_matrices=False)
    return {"singular_values": S[:k].numpy(), "components": Vt[:k].numpy(),
            "mean": mean.cpu().numpy().ravel()}

rp = randomized_pca(X, 10, device=device)
exact = np.linalg.svd(X - X.mean(axis=0), compute_uv=False)[:10]
{"max relative error of singular values": float(np.max(np.abs(rp["singular_value
```

Out[19]:
```
{'max relative error of singular values': 9.67290723775492e-06}
```

A larger synthetic problem: $n = 20\,000$ observations in $d = 2000$ dimensions, generated as a rank-30 signal with decaying strengths plus isotropic noise. The cell compares the CPU with the available accelerator (on a CPU-only machine both numbers are CPU timings).

In [20]:
```
n_big, d_big, r_big = 20_000, 2_000, 30
strengths = np.linspace(5.0, 1.0, r_big)[:, None]
L = rng.standard_normal((n_big, r_big)) @ (strengths * rng.standard_normal((r_bi
X_big = (L + 0.5 * rng.standard_normal((n_big, d_big))).astype(np.float32)

def time_randomized_pca(X, k, device, repeats=3):
    randomized_pca(X, k, device=device)           # warm-up (kernel compilation,
    synchronize(device)
    start = time.perf_counter()
    for _ in range(repeats):
        randomized_pca(X, k, device=device)
    synchronize(device)
    return (time.perf_counter() - start) / repeats

{"cpu_seconds": time_randomized_pca(X_big, 30, torch.device("cpu")),
 f"{device.type}_seconds": time_randomized_pca(X_big, 30, device)}
```

Out[20]:
```
{'cpu_seconds': 0.059198986001623176, 'mps_seconds': 0.04009236133424565}
```

In [21]:
```
rp_big = randomized_pca(X_big, 30, device=device)
sk_big = PCA(n_components=30, svd_solver="randomized", random_state=0).fit(X_big
{"max relative difference vs sklearn": float(np.max(np.abs(rp_big["singular_valu
                                                    / sk_big.singular_values_)))}
```

Out[21]:
```
{'max relative difference vs sklearn': 4.94017876606589e-07}
```

2.6 Neighbourhood-preserving maps: t-SNE and UMAP

PCA preserves large variances, i.e. the global shape. For *visualising* clusters we often care about the opposite: which points are each other's neighbours. Nonlinear embeddings optimise exactly that.

t-SNE (t-Distributed Stochastic Neighbor Embedding)

> van der Maaten & Hinton, 2008

The idea in one sentence. Describe the neighbourhood of every point as a *probability distribution* over the other points ("if x_i picked a neighbour at random, how likely would it pick x_j?"), do the same in the plane, and move the points y_i until the two sets of distributions agree.

The goal

- We want to place each data point $x_i \in \mathbb{R}^D$ at a position $y_i \in \mathbb{R}^2$ in the plane so that *points which are neighbours* in the original space remain neighbours in the picture.

- We do not care about preserving large distances, only about who is close to whom.

The plan

1. Turn high-dimensional distances into neighbour probabilities $p_{j|i}$. This requires a width parameter for each point, set through a quantity called **perplexity**.
2. Symmetrise them into joint probabilities p_{ij}.
3. Define analogous probabilities q_{ij} for the points y_i in the plane.
4. Measure the mismatch between P and Q with the Kullback–Leibler divergence and minimise it by gradient descent over the positions y_i.

Step 1 — neighbour probabilities in the original space. Place a Gaussian of width σ_i at x_i and let x_i choose a neighbour j with probability proportional to the Gaussian density at x_j:

$$p_{j|i} = \frac{\exp\!\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{l \neq i} \exp\!\left(-\|x_i - x_l\|^2 / 2\sigma_i^2\right)}, \qquad p_{i|i} = 0.$$

For each i the numbers $p_{j|i}$ are non-negative and sum to 1 over j, so $P_i = (p_{1|i}, \ldots, p_{n|i})$ is a probability distribution: the *neighbour distribution* of point i. Close points get large probabilities, distant ones practically zero.

The role of σ_i. The width decides how many points count as neighbours.

- Small σ_i: almost all probability goes to the single nearest point.
- Large σ_i: the probability spreads over many points, eventually almost uniformly over all of them.

One global σ would not work, because density varies across the data:

- in a dense cluster a given σ covers hundreds of points
- in a sparse region the same σ may cover none
- a separate σ_i chosen for every point so that each one has *the same effective number of neighbours*
- to make "effective number of neighbours" precise, entropy is needed

Entropy. The (Shannon) entropy of a discrete distribution P_i is

$$H(P_i) = -\sum_j p_{j|i} \ln p_{j|i} \qquad (\text{with } 0 \ln 0 = 0).$$

It measures how spread out the distribution is:

- if all mass sits on one point, $H = 0$;
- if the mass is spread uniformly over k points, each with probability $1/k$, then $H = -k \cdot \frac{1}{k} \ln \frac{1}{k} = \ln k$;
- in general $0 \leq H(P_i) \leq \ln(n-1)$, with the maximum attained by the uniform distribution.

Perplexity. The perplexity of P_i is the exponential of its entropy,

$$\mathrm{Perp}(P_i) = e^{H(P_i)}.$$

- a distribution that is uniform over k points has perplexity exactly k (see above)
- for a general distribution, perplexity is the number of points over which a *uniform* distribution would be equally spread out
- it is read as the **effective number of neighbours**

Calibration. The user chooses one target perplexity $\mathcal{P}$, typically between 5 and 50, and for every point we solve

$$\mathrm{Perp}(P_i) = \mathcal{P} \quad \text{for } \sigma_i.$$

- the entropy increases monotonically with $\sigma_i \rightarrow$ the equation has a unique solution
- bisection:
  - if the entropy is too high, decrease σ_i;
  - if too low, increase it
- points in dense regions end up with a small σ_i, points in sparse regions with a large one
- in the following code, we work with the precision $\beta_i = 1/(2\sigma_i^2)$ instead of σ_i, which only reverses the direction of the bisection

In [22]:
```
def calibrate_affinities(X, perplexity=30.0, n_steps=64):
    # Conditional p_{j|i} with per-point precision found by bisection on the ent
    X = np.asarray(X, dtype=np.float64)
    n = X.shape[0]
    sq = np.sum(X * X, axis=1)
    D = np.maximum(sq[:, None] + sq[None, :] - 2.0 * X @ X.T, 0.0)
    off = ~np.eye(n, dtype=bool)
    Dmin = np.where(off, D, np.inf).min(axis=1, keepdims=True)
    Dm = np.where(off, D - Dmin, 0.0)             # shift for numerical stabilit
    target = np.log(perplexity)
    beta, lo, hi = np.ones(n), np.zeros(n), np.full(n, np.inf)
    for _ in range(n_steps):
        W = np.exp(-beta[:, None] * Dm) * off
        Zsum = W.sum(axis=1)
        H = np.log(Zsum) + beta * (W * Dm).sum(axis=1) / Zsum
        too_flat = H > target                     # entropy too high: increase p
        lo = np.where(too_flat, beta, lo)
        hi = np.where(too_flat, hi, beta)
        beta = np.where(np.isinf(hi), 2.0 * beta, 0.5 * (lo + hi))
    P_cond = W / Zsum[:, None]
    return P_cond, float(np.max(np.abs(H - target)))

P_cond, entropy_error = calibrate_affinities(X / 16.0, perplexity=30.0)
{"max |H - ln(perplexity)|": entropy_error}
```

Out[22]:
```
{'max |H - ln(perplexity)|': 1.7763568394002505e-15}
```

The entropy in the above code is not computed from its definition but from an equivalent formula (which avoids taking logarithms of tiny probabilities): since $\ln p_{j|i} = -\beta_i d_{ij} - \ln Z_i$, where $d_{ij} = \|x_i - x_j\|^2$ and Z_i is the normalising sum,

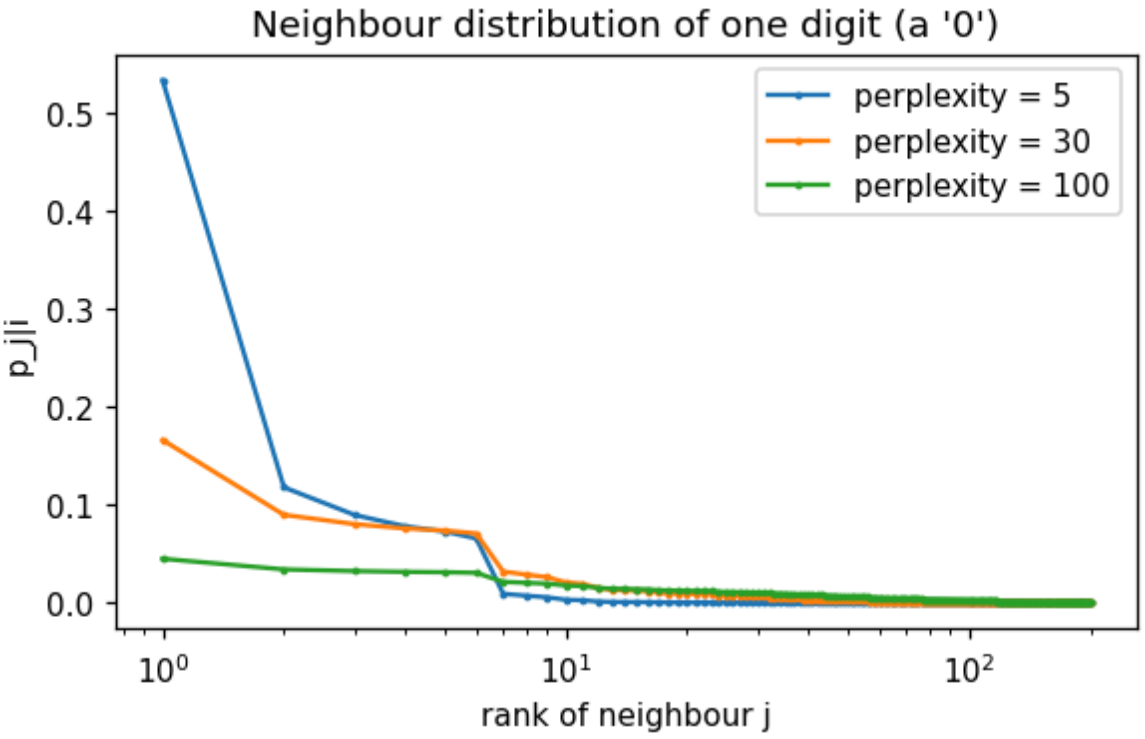
$$H(P_i) = \ln Z_i + \beta_i \sum_j p_{j|i}\, d_{ij},$$

Moreover, every row of distances is shifted by its minimum before exponentiation; the shift cancels between the numerator and the denominator of $p_{j|i}$, but it prevents all exponentials from underflowing to zero.

The cell below shows what the perplexity does to one particular digit: its neighbour distribution, sorted from the most to the least probable neighbour, for three values of $\mathcal{P}$.

In [23]:
```
i = 0
fig, ax = plt.subplots(figsize=(6, 3.4))
perplexity_summary = {}
for perp in [5, 30, 100]:
    P_perp, _ = calibrate_affinities(X / 16.0, perplexity=perp)
    p_i = np.sort(P_perp[i])[::-1]
    nz = p_i[p_i > 0]
    perplexity_summary[perp] = {
        "exp(entropy)": round(float(np.exp(-np.sum(nz * np.log(nz))))), 2),
        "largest p_j|i": round(float(p_i[0]), 3),
        "neighbours holding 90% of mass": int(np.searchsorted(np.cumsum(p_i), 0.
    }
    ax.plot(np.arange(1, 201), p_i[:200], marker=".", ms=3, label=f"perplexity =
ax.set_xscale("log")
ax.set_xlabel("rank of neighbour j")
ax.set_ylabel("p_j|i")
ax.set_title(f"Neighbour distribution of one digit (a '{y[i]}')")
ax.legend()
plt.show()

perplexity_summary
```



Out[23]:
```
{5: {'exp(entropy)': 5.0,
  'largest p_j|i': 0.534,
  'neighbours holding 90% of mass': 6},
 30: {'exp(entropy)': 30.0,
  'largest p_j|i': 0.166,
  'neighbours holding 90% of mass': 36},
 100: {'exp(entropy)': 100.0,
  'largest p_j|i': 0.045,
  'neighbours holding 90% of mass': 94}}
```

Step 2 — symmetrise. The conditional probabilities are not symmetric: $p_{j|i} \neq p_{i|j}$ in general, because $\sigma_i \neq \sigma_j$. t-SNE uses one joint distribution over all pairs,

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}, \qquad \sum_{i \neq j} p_{ij} = 1.$$

Besides symmetry this protects outliers. If x_i is far from everything, all $p_{i|j}$ are tiny, but its own $p_{j|i}$ still sum to 1, so $\sum_j p_{ij} \geq 1/(2n)$: every point keeps some pull on its neighbours and cannot drift off arbitrarily.

Step 3 — probabilities in the plane. For the map points $y_i \in \mathbb{R}^2$ we define analogous joint probabilities, but with a **Student-t kernel with one degree of freedom** (a Cauchy kernel) instead of a Gaussian, and with no per-point width:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}, \qquad q_{ii} = 0.$$

Why a different kernel: the crowding problem.

- two dimensions have far less room than D dimensions:
  - in $\mathbb{R}^{10}$ one can place 11 points that are all at the same distance from each other
  - in the plane at most 3
- a point in high dimension has many more neighbours at a *moderate* distance than a point in the plane can accommodate
- if we used a Gaussian in the plane as well, all these moderately distant points would be squeezed into a small disc around y_i
- the clusters would smear into each other
- the Student-t kernel decays like $1/r^2$ (*polynomially*)
  - to reproduce a small probability q_{ij} it is enough to put y_j moderately far away
  - the cost of moving it even further is small
  - moderately dissimilar points can thus be pushed out, which opens visible gaps between clusters

	original space	**plane**
kernel	Gaussian, $e^{-d^2/2\sigma_i^2}$	Student-t, $(1+d^2)^{-1}$
width	σ_i per point, set by perplexity	fixed
normalisation	per point, then symmetrised	over all pairs

Step 4 — the objective. The mismatch between the two joint distributions is measured by the Kullback–Leibler divergence

$$C(y_1, \ldots, y_n) = \mathrm{KL}(P \| Q) = \sum_{i \neq j} p_{ij} \ln \frac{p_{ij}}{q_{ij}}.$$

It is non-negative and zero only when $Q = P$. The p_{ij} are fixed by the data; only the positions y_i are variables.

KL is asymmetric, and this decides what t-SNE preserves.

- p_{ij} large, q_{ij} small (true neighbours placed far apart): the term $p_{ij} \ln(p_{ij}/q_{ij})$ is large. *Heavily penalised.*
- p_{ij} small, q_{ij} large (non-neighbours placed close): the term is weighted by the small p_{ij}. *Cheap.*

t-SNE therefore works hard to keep neighbourhoods intact and says little about large distances, cluster sizes or the relative position of clusters.

The gradient. A direct (if somewhat tedious) computation gives

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij}) \left(1 + \|y_i - y_j\|^2\right)^{-1} (y_i - y_j).$$

Read it as a system of springs between y_i and every other point y_j, acting along $y_i - y_j$. Gradient descent moves y_i against the gradient, so:

- if $p_{ij} > q_{ij}$, the points are drawn *together* (the pair should be closer than it is);
- if $p_{ij} < q_{ij}$, they are pushed *apart*;
- the factor $\left(1 + \|y_i - y_j\|^2\right)^{-1}$ weakens the interaction between points that are already far apart, so distant clusters do not keep pulling on each other.

Step 5 — optimisation. The objective is non-convex, so the result depends on the initialisation and on a few heuristics from the original paper, all present in the implementation below:

- *Initialisation:* tiny random positions, $y_i \sim \mathcal{N}(0, 10^{-4} I)$, so that no structure is imposed at the start.
- *Early exaggeration:* for the first 250 iterations all p_{ij} are multiplied by 12. The attractive forces dominate, and points of the same cluster gather into tight groups that can then move as units, instead of getting stuck between clusters.
- *Momentum* 0.5, later 0.8: each step adds a fraction of the previous step, which smooths and speeds up descent.
- *Adaptive gains:* the step size of each coordinate grows while the gradient keeps its sign and shrinks when it oscillates.

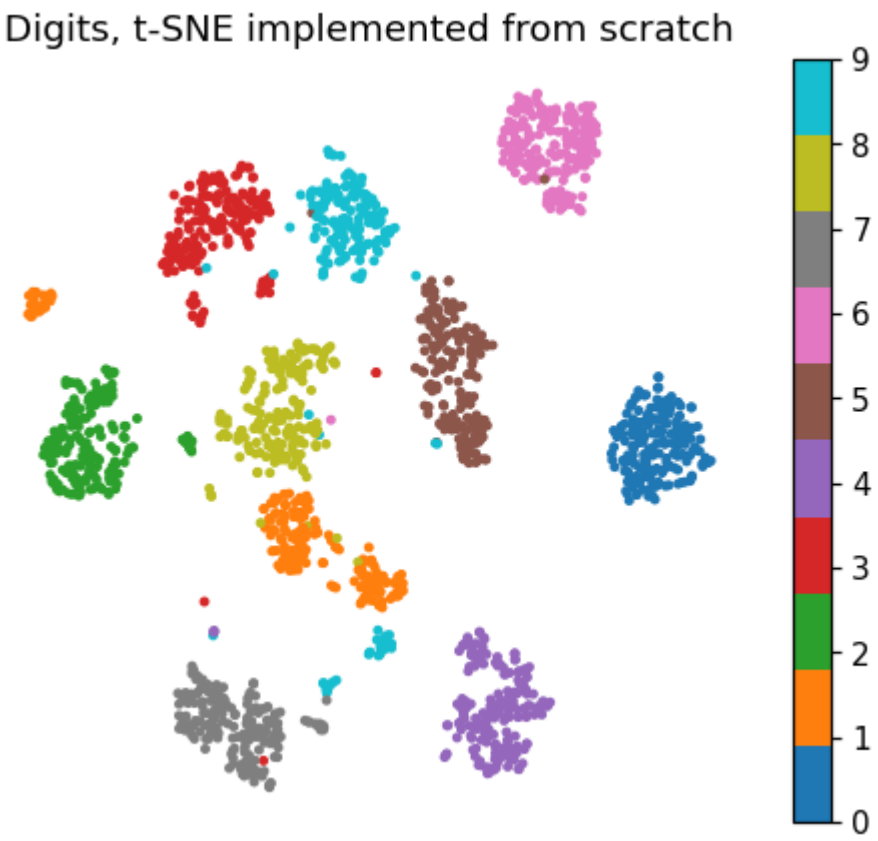
Cost. In the exact form every iteration involves all n^2 pairs. These are dense matrix operations that map well onto a GPU; for large n, approximations (Barnes–Hut, as in scikit-learn) reduce the cost to $O(n \log n)$.

In [24]:
```
def tsne(P_cond, n_iter=1000, exaggeration=12.0, n_exaggeration=250, lr=None, se
    device = get_device() if device is None else device
    n = P_cond.shape[0]
    P = np.maximum((P_cond + P_cond.T) / (2.0 * n), 1e-12)
    P = torch.as_tensor(P, dtype=torch.float32, device=device)
    off = 1.0 - torch.eye(n, device=device)
    gen = torch.Generator().manual_seed(seed)
    Y = (1e-4 * torch.randn(n, 2, generator=gen)).to(device)
    velocity = torch.zeros_like(Y)
    gains = torch.ones_like(Y)
    lr = max(n / exaggeration / 4.0, 50.0) if lr is None else lr
    for it in range(n_iter):
        ex = exaggeration if it < n_exaggeration else 1.0
        momentum = 0.5 if it < n_exaggeration else 0.8
        sq = (Y * Y).sum(dim=1, keepdim=True)
        num = off / (1.0 + (sq + sq.T - 2.0 * Y @ Y.T).clamp_min(0.0))   # Stude
        Q = (num / num.sum()).clamp_min(1e-12)
        M = (ex * P - Q) * num
        grad = 4.0 * (M.sum(dim=1, keepdim=True) * Y - M @ Y)              # the
        same_sign = torch.sign(grad) == torch.sign(velocity)
        gains = torch.where(same_sign, gains * 0.8, gains + 0.2).clamp_min(0.01)
        velocity = momentum * velocity - lr * gains * grad
        Y = Y + velocity
        Y = Y - Y.mean(dim=0, keepdim=True)
    kl = float((P * torch.log(P / Q)).sum())
    return Y.cpu().numpy(), kl

Y_ours, kl_ours = tsne(P_cond, device=device)

plt.figure(figsize=(5.5, 4.5))
sc = plt.scatter(Y_ours[:, 0], Y_ours[:, 1], c=y, cmap="tab10", s=5)
plt.colorbar(sc, ticks=range(10))
plt.title("Digits, t-SNE implemented from scratch")
plt.axis("off")
plt.show()

{"final KL(P||Q)": kl_ours}
```



Out[24]:
```
{'final KL(P||Q)': 0.6860722303390503}
```

Measuring neighbourhood preservation. The *trustworthiness* (Venna & Kaski, 2001) of an embedding penalises points that appear among the K nearest neighbours in the map but not in the original space:

$$T(K) = 1 - \frac{2}{nK(2n - 3K - 1)} \sum_{i=1}^{n} \sum_{j \in \mathcal{U}_i^K} \big(r(i,j) - K\big),$$

where $\mathcal{U}_i^K$ are those intruding neighbours and $r(i,j)$ is the rank of j among the neighbours of i in the original space. $T = 1$ means no intruders. We compare our implementation with scikit-learn's and with the PCA plane.

In [25]:
```
from sklearn.manifold import TSNE, trustworthiness

Y_sk = TSNE(perplexity=30, init="random", random_state=0).fit_transform(X)
Y_pca = PCA(n_components=2).fit_transform(X)

{"trustworthiness (K=10), PCA": round(float(trustworthiness(X, Y_pca, n_neighbor
 "trustworthiness (K=10), t-SNE ours": round(float(trustworthiness(X, Y_ours, n_
 "trustworthiness (K=10), t-SNE sklearn": round(float(trustworthiness(X, Y_sk, n
```

Out[25]:
```
{'trustworthiness (K=10), PCA': 0.83,
 'trustworthiness (K=10), t-SNE ours': 0.992,
 'trustworthiness (K=10), t-SNE sklearn': 0.9933}
```

The same CPU vs accelerator comparison for the $O(n^2)$ t-SNE loop (300 iterations to keep it short):

In [29]:
```
def time_tsne(P_cond, device, n_iter=300):
    tsne(P_cond, n_iter=10, device=device)        # warm-up
    synchronize(device)
    start = time.perf_counter()
    tsne(P_cond, n_iter=n_iter, device=device)
    synchronize(device)
    return time.perf_counter() - start

{"cpu_seconds": time_tsne(P_cond, torch.device("cpu"),3000),
 f"{device.type}_seconds": time_tsne(P_cond, device, 3000)}
```

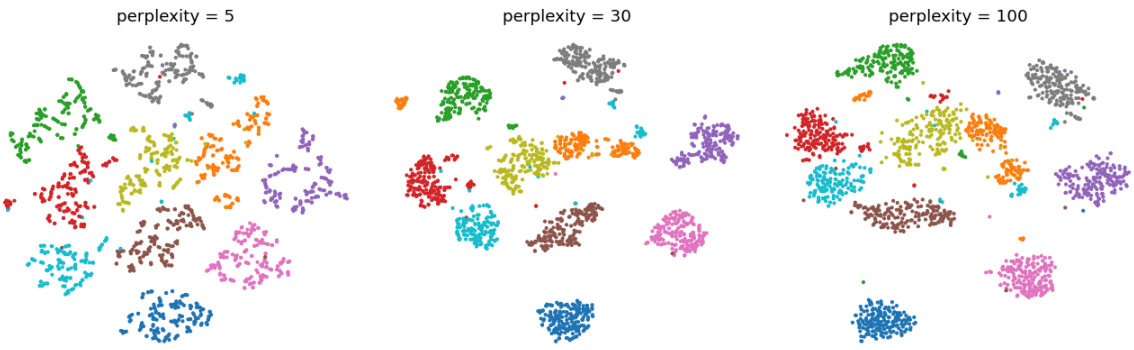
Out[29]:
```
{'cpu_seconds': 9.16017266601557, 'mps_seconds': 9.08760429098038}
```

Why the accelerator barely helps here.

- each t-SNE iteration consists of about fifteen operations on $n \times n$ matrices (13 MB each for the digits)
- almost every one of them performs only one or two arithmetic operations per element it reads or writes
- even `Y @ Y.T` has inner dimension 2, so it is essentially an outer product
- such code is **memory-bound**: its speed is set by how fast bytes move between memory and the processor, not by how fast the processor computes
- on Apple silicon the CPU and the GPU share the same unified memory:
  - a memory-bound loop runs at about the same speed on both

Perplexity is a scale parameter. Small values see only very local structure (and fragment clusters); large values merge them. There is no single correct value, so it is good practice to look at several.

In [25]:
```
fig, axes = plt.subplots(1, 3, figsize=(12, 3.8))
for ax, perp in zip(axes, [5, 30, 100]):
    Yp = TSNE(perplexity=perp, init="pca", random_state=0).fit_transform(X)
    ax.scatter(Yp[:, 0], Yp[:, 1], c=y, cmap="tab10", s=3)
    ax.set_title(f"perplexity = {perp}")
    ax.axis("off")
plt.tight_layout()
plt.show()
```



UMAP (*Uniform Manifold Approximation and Projection*)

> McInnes, Healy & Melville, 2018

The idea in one sentence. Build a weighted graph that says *how strongly each pair of points is connected* in the original space, build the analogous graph for the points in the plane, and move the points until the two graphs agree.

The goal

- same as for t-SNE: place the points in the plane so that neighbours stay neighbours

The plan.

1. Find the k nearest neighbours of every point and give each of these edges a weight in $(0, 1]$, using a per-point offset ρ_i and scale σ_i.
2. Symmetrise the directed weights with a *fuzzy union*.
3. Define edge weights between points in the plane with a kernel controlled by `min_dist`.
4. Minimise the binary cross-entropy between the two graphs by stochastic gradient descent with negative sampling.

The main difference from t-SNE:

- t-SNE compares *probability distributions* (everything sums to 1, so all pairs are coupled)
- UMAP compares *independent edge weights* (each pair is judged on its own)

Step 1 — a weighted k-nearest-neighbour graph. Fix the number of neighbours k (`n_neighbors`, default 15) and find, for every point x_i, the set $N_k(i)$ of its nearest neighbours. Only these edges receive a weight; all other pairs are treated as unconnected. For $j \in N_k(i)$,

$$w_{j|i} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right), \qquad \rho_i = \min_{j \in N_k(i)} d(x_i, x_j),$$

and $w_{j|i} = 0$ otherwise. Note that d is the distance itself, not its square.

The offset ρ_i. Subtracting the distance to the nearest neighbour has two effects.

- *Local connectivity.* The nearest neighbour always gets weight $e^0 = 1$, so no point is ever isolated, however sparse its region.
- *It removes the common part of the distances.* Section 2.2(b) showed that in high dimension all distances from a point tend to be large and similar. What distinguishes neighbours is the *excess* $d(x_i, x_j) - \rho_i$ over the closest one, and that is what the weight measures.

The scale σ_i plays the role of t-SNE's σ_i. It is chosen separately for every point by bisection so that the weights sum to a fixed value:

$$\sum_{j \in N_k(i)} w_{j|i} = \log_2 k.$$

As with perplexity, this makes points in dense and sparse regions look alike: each point "sees" the same total amount of connection, measured in its own units. The target $\log_2 k$ is a heuristic choice of the authors, not a deep result; what matters is that it is the same for all points and grows with k.

Weights, not probabilities. The $w_{j|i}$ lie in $(0, 1]$ but do *not* sum to 1. Each one is read as the probability that the edge $i \to j$ exists, independently of the others. This is what allows UMAP to drop the global normalisation of t-SNE.

How to think of `n_neighbors`. It is the counterpart of perplexity:

- small values look only at very local structure
- large values let each point feel a bigger part of the data set
- unlike perplexity it is also a hard cut-off, since pairs outside the k-NN graph carry no attraction at all

In [31]:
```
from sklearn.neighbors import NearestNeighbors

def umap_weights(X, n_neighbors=15, n_steps=64):
    # Directed kNN weights w_{j|i} with UMAP's per-point offset rho_i and scale
    nn = NearestNeighbors(n_neighbors=n_neighbors).fit(X)
    dist, idx = nn.kneighbors(X)                    # column 0 is the point itself
    dist, idx = dist[:, 1:], idx[:, 1:]
    rho = dist[:, 0]                                # distance to the nearest neig
    excess = np.maximum(dist - rho[:, None], 0.0)
    target = np.log2(n_neighbors)
    sigma, lo, hi = np.ones(len(X)), np.zeros(len(X)), np.full(len(X), np.inf)
    for _ in range(n_steps):
        total = np.exp(-excess / sigma[:, None]).sum(axis=1)
        too_wide = total > target                   # too much weight: shrink sigm
        hi = np.where(too_wide, sigma, hi)
        lo = np.where(too_wide, lo, sigma)
        sigma = np.where(np.isinf(hi), 2.0 * sigma, 0.5 * (lo + hi))
    W = np.exp(-excess / sigma[:, None])
    return W, idx, rho, sigma

W_dir, knn_idx, rho, sigma = umap_weights(X, n_neighbors=15)

check = {"row sums equal log2(15)": bool(np.allclose(W_dir.sum(axis=1), np.log2(
         "smallest nearest-neighbour weight": float(W_dir[:, 0].min()),
         "sigma range": (round(float(sigma.min()), 3), round(float(sigma.max()),
try:                                                # compare with the umap-learn
    from umap.umap_ import smooth_knn_dist
    d_ref, _ = NearestNeighbors(n_neighbors=15).fit(X).kneighbors(X)
    sigma_ref, rho_ref = smooth_knn_dist(d_ref.astype(np.float32), 15.0)
    check["max relative difference of sigma vs umap-learn"] = float(np.max(np.ab
except ImportError:
    pass
check
```

Out[31]:
```
{'row sums equal log2(15)': True,
 'smallest nearest-neighbour weight': 1.0,
 'sigma range': (0.378, 10.444),
 'max relative difference of sigma vs umap-learn': 5.686386509123507e-06}
```

Step 2 — symmetrise with a fuzzy union. The directed weights are not symmetric: j may be among the nearest neighbours of i but not vice versa, and $\sigma_i \neq \sigma_j$. Reading $w_{j|i}$ as the probability that the edge exists "according to i", and $w_{i|j}$ as the probability according to j, UMAP keeps the edge if *either* endpoint sees it. For independent events,

$$w_{ij} = P(A \cup B) = w_{j|i} + w_{i|j} - w_{j|i}\, w_{i|j}.$$

This is a *probabilistic OR* (a t-conorm in fuzzy-set language). It differs from t-SNE's average: an edge seen strongly by one endpoint keeps a strong weight even if the other endpoint does not see it at all. The result is a sparse symmetric graph with about nk edges, all weights in $(0, 1]$.

In [32]:
```
from scipy.sparse import csr_matrix

n_pts = len(X)
rows = np.repeat(np.arange(n_pts), W_dir.shape[1])
A_dir = csr_matrix((W_dir.ravel(), (rows, knn_idx.ravel())), shape=(n_pts, n_pts
A_T = A_dir.T.tocsr()
W_graph = A_dir + A_T - A_dir.multiply(A_T)       # fuzzy union

{"directed edges": int(A_dir.nnz), "edges after union": int(W_graph.nnz),
 "symmetric": bool(abs(W_graph - W_graph.T).max() < 1e-12),
 "density of the graph": round(W_graph.nnz / n_pts**2, 4)}
```

Out[32]:
```
{'directed edges': 25158,
 'edges after union': 34234,
 'symmetric': True,
 'density of the graph': 0.0106}
```

Step 3 — edge weights in the plane. For map points $y_i \in \mathbb{R}^2$ UMAP uses

$$v_{ij} = \frac{1}{1 + a\,\|y_i - y_j\|^{2b}},$$

with **no normalisation** over pairs. For $a = b = 1$ this is exactly t-SNE's Student-t kernel before normalisation, so it has the same heavy tail that counters the crowding problem.

The constants a, b are not chosen by hand. They are fitted by least squares so that v approximates the curve

$$\psi(r) = \begin{cases} 1, & r \leq \texttt{min_dist}, \\ \exp\big(-(r - \texttt{min_dist})\big), & r > \texttt{min_dist}, \end{cases}$$

where `min_dist` (default 0.1) is a user parameter. It says *how close points may sit in the picture*: up to that distance an edge counts as fully present, so there is no incentive to pull points even closer.

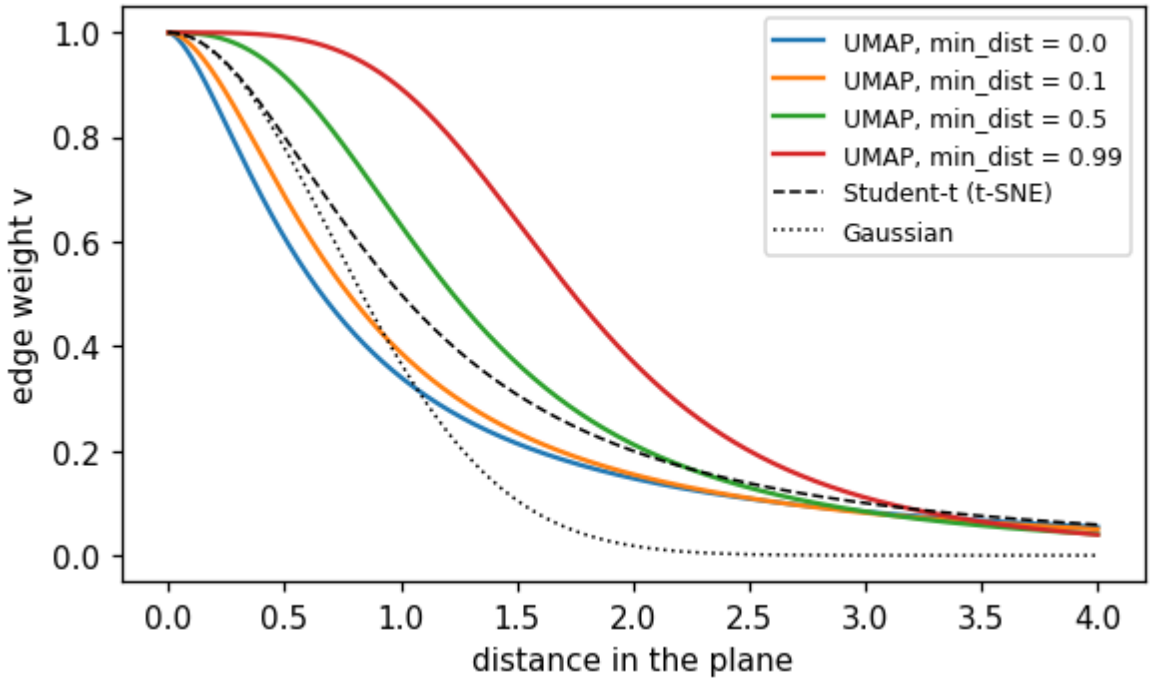
- Small `min_dist`: points within a cluster collapse into tight clumps; good for finding clusters.
- Large `min_dist`: points spread out evenly; good for seeing the internal structure of a cluster.

`min_dist` affects only the picture, not the graph built in Steps 1–2.

In [34]:
```
from umap.umap_ import find_ab_params

r = np.linspace(0, 4, 400)
fig, ax = plt.subplots(figsize=(6, 3.4))
ab_params = {}
for md_ in [0.0, 0.1, 0.5, 0.99]:
    a, b = find_ab_params(1.0, md_)
    ab_params[md_] = (round(float(a), 3), round(float(b), 3))
    ax.plot(r, 1.0 / (1.0 + a * r ** (2 * b)), label=f"UMAP, min_dist = {md_}")
ax.plot(r, 1.0 / (1.0 + r**2), "k--", lw=1, label="Student-t (t-SNE)")
ax.plot(r, np.exp(-r**2), "k:", lw=1, label="Gaussian")
ax.set_xlabel("distance in the plane")
ax.set_ylabel("edge weight v")
ax.legend(fontsize=8)
plt.show()

{"(a, b) for each min_dist": ab_params}
```



Out[34]:
```
{'(a, b) for each min_dist': {0.0: (1.933, 0.79),
  0.1: (1.577, 0.895),
  0.5: (0.583, 1.334),
  0.99: (0.119, 1.916)}}
```

Step 4 — the objective: binary cross-entropy. Every pair (i, j) is treated as a binary variable "the edge exists", with probability w_{ij} in the data and v_{ij} in the picture. The mismatch is the sum of binary cross-entropies (KL divergences between two Bernoulli distributions):

$$C = \sum_{i \neq j} \left[w_{ij} \ln \frac{w_{ij}}{v_{ij}} + (1 - w_{ij}) \ln \frac{1 - w_{ij}}{1 - v_{ij}} \right].$$

Dropping the terms that do not depend on the positions y_i,

$$C = \underbrace{-\sum_{i \neq j} w_{ij} \ln v_{ij}}_{\text{attraction}} \underbrace{-\sum_{i \neq j} (1 - w_{ij}) \ln(1 - v_{ij})}_{\text{repulsion}} + \text{const.}$$

- **Attraction.** Non-zero only for the $\approx nk$ edges of the graph. It is small when $v_{ij} \approx 1$, i.e. when connected points sit close together.
- **Repulsion.** Involves all pairs, almost all of which have $w_{ij} = 0$; for them the term is $-\ln(1 - v_{ij})$, which is small only when $v_{ij} \approx 0$, i.e. when unconnected points sit far apart.

Compare with t-SNE:

- there, repulsion appears only *implicitly*, through the normalisation of Q
  - that normalisation couples all n^2 pairs
- UMAP has an *explicit* repulsive term, and each pair contributes separately, which is what makes sampling possible

The gradient as springs. With $D_{ij} = \|y_i - y_j\|^2$, the gradients of a single attractive and a single repulsive term with respect to y_i are

$$\frac{2ab\, D_{ij}^{b-1}}{1 + aD_{ij}^{b}}\,(y_i - y_j) \qquad \text{and} \qquad -\frac{2b}{(\varepsilon + D_{ij})(1 + aD_{ij}^{b})}\,(y_i - y_j),$$

with a small ε (0.001) to avoid division by zero. Descending along them moves connected points towards each other and unconnected points apart, as in t-SNE.

Optimisation: SGD with negative sampling. Summing the repulsion over all n^2 pairs would cost as much as t-SNE. UMAP instead runs stochastic gradient descent over *edges*:

1. sample an edge (i, j) of the graph, with frequency proportional to w_{ij}, and apply the attractive update;
2. draw a few random points l (`negative_sample_rate` , default 5), treat them as unconnected to i, and apply the repulsive update.

One epoch therefore costs $O(nk)$, and with an approximate nearest-neighbour search (NN-descent) to build the graph, the whole method scales roughly linearly in n. The learning rate decays linearly over the epochs (500 for small data sets, 200 for large ones).

Initialisation.

- by default the points do not start at random
- instead, the *spectral embedding* of the graph is used:
  - the eigenvectors of its graph Laplacian with the smallest non-zero eigenvalues
  - a globally sensible starting layout for optimization
- Kobak & Linderman (2021):
  - much of UMAP's better global structure comes from this initialisation
  - t-SNE initialised with PCA behaves similarly

A caveat.

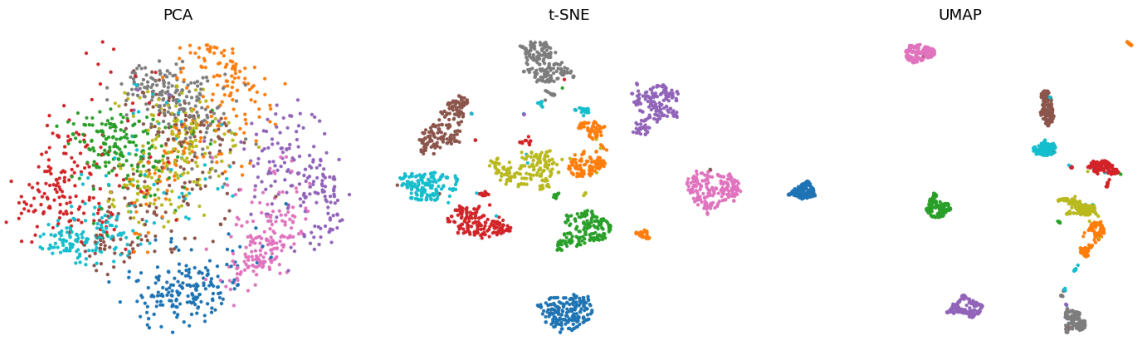
- because of negative sampling, the function UMAP actually minimises is not exactly the cross-entropy above
- the effective repulsion is considerably weaker (Damrich & Hamprecht, NeurIPS 2021)

- this is why UMAP clusters tend to be tighter than the formula suggests

In [35]:
```
try:
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        import umap
        Y_umap = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=0).fit_tra
except ImportError:
    Y_umap = None                                   # pip install umap-learn

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
embeddings = {"PCA": Y_pca, "t-SNE": Y_sk, "UMAP": Y_umap}
for ax, (name, E) in zip(axes, embeddings.items()):
    if E is not None:
        ax.scatter(E[:, 0], E[:, 1], c=y, cmap="tab10", s=3)
    ax.set_title(name)
    ax.axis("off")
plt.tight_layout()
plt.show()

{name: round(float(trustworthiness(X, E, n_neighbors=10)), 4)
 for name, E in embeddings.items() if E is not None}
```



Out[35]:
```
{'PCA': 0.83, 't-SNE': 0.9933, 'UMAP': 0.988}
```

t-SNE and UMAP side by side.

	t-SNE	UMAP
neighbourhood size	perplexity, soft (all pairs)	`n_neighbors`, hard k-NN cut-off
high-dimensional weights	Gaussian on squared distance, σ_i per point	exponential on distance minus ρ_i, σ_i per point
symmetrisation	average $(p_{j\|i} + p_{i\|j})/2n$	fuzzy union $w + w' - ww'$
weights in the plane	Student-t, normalised over all pairs	$(1 + aD^b)^{-1}$, not normalised; a, b from `min_dist`
objective	$\mathrm{KL}(P\|Q)$	binary cross-entropy, explicit repulsion
optimisation	full gradient, $O(n^2)$ (Barnes–Hut $O(n \log n)$)	SGD over edges with negative sampling, $\approx O(n)$
initialisation	random or PCA	spectral embedding of the graph

	t-SNE	UMAP
new points	not supported	`transform` embeds them into an existing map

Hardware.

- `umap-learn` runs on the CPU, with Numba-compiled kernels (multithreaded unless a `random_state` is fixed)
- A GPU implementation exists in RAPIDS cuML, for CUDA only; there is none for MPS
  - a gap not worth filling here: the work consists of many tiny, random, sparse updates, which is exactly the memory-bound pattern that gave t-SNE no speed-up on the accelerator

3. Real-world examples

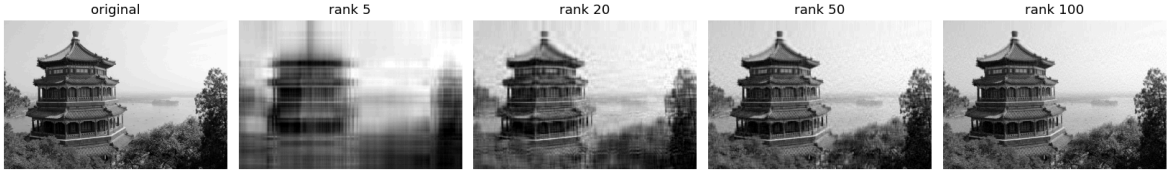
3.1 Low-rank image compression (Eckart–Young in action)

A grey-scale photograph is a matrix. Its singular values typically decay fast, so a truncated SVD stores $k(m+n+1)$ numbers instead of mn. The image below ships with scikit-learn (a photograph of the Summer Palace in Beijing, released under a CC licence).

In [27]:
```
img = load_sample_image("china.jpg").astype(np.float64) / 255.0
gray = img.mean(axis=2)                              # (427, 640)
U_i, S_i, Vt_i = np.linalg.svd(gray, full_matrices=False)
m, n_cols = gray.shape

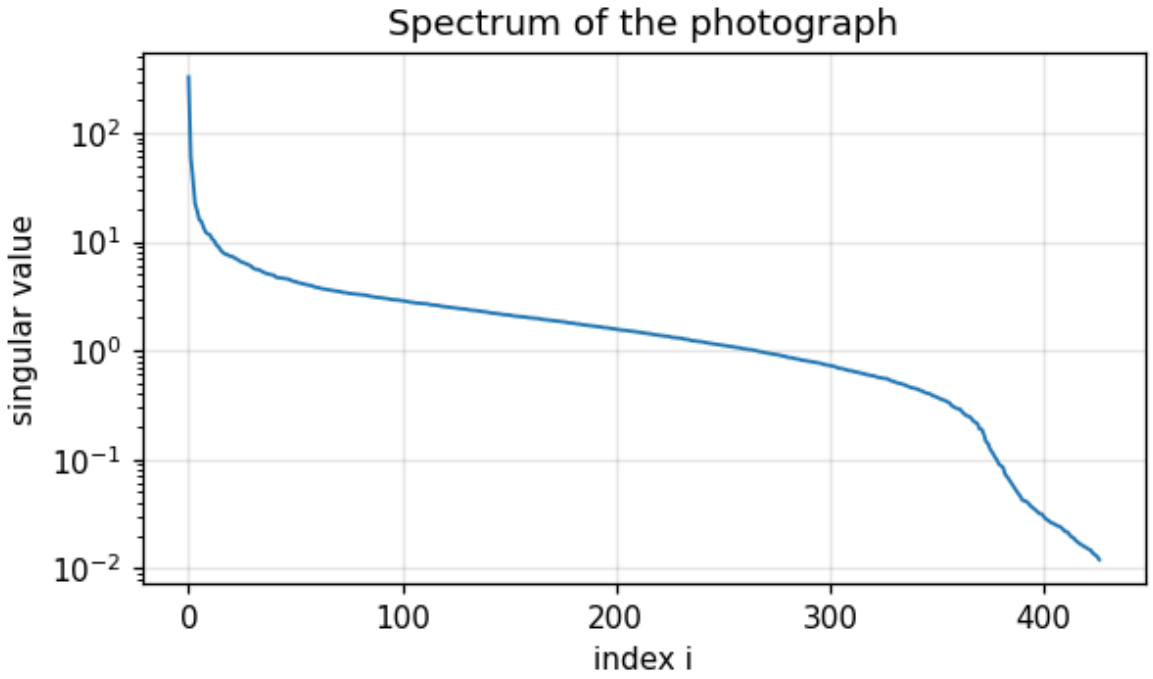
ks_img = [5, 20, 50, 100]
fig, axes = plt.subplots(1, len(ks_img) + 1, figsize=(15, 2.8))
axes[0].imshow(gray, cmap="gray")
axes[0].set_title("original")
summary = {}
for ax, k in zip(axes[1:], ks_img):
    approx = (U_i[:, :k] * S_i[:k]) @ Vt_i[:k]
    rel_err = np.linalg.norm(gray - approx) / np.linalg.norm(gray)
    storage = k * (m + n_cols + 1) / (m * n_cols)
    summary[k] = {"relative error": round(float(rel_err), 3), "storage fraction"
    ax.imshow(np.clip(approx, 0, 1), cmap="gray")
    ax.set_title(f"rank {k}")
for ax in axes:
    ax.axis("off")
plt.tight_layout()
plt.show()

summary
```



Out[27]:
```
{5: {'relative error': 0.184, 'storage fraction': 0.02},
 20: {'relative error': 0.136, 'storage fraction': 0.078},
 50: {'relative error': 0.103, 'storage fraction': 0.195},
 100: {'relative error': 0.074, 'storage fraction': 0.391}}
```

In [28]:
```
plt.figure(figsize=(6, 3.2))
plt.semilogy(S_i, lw=1.2)
plt.xlabel("index i")
plt.ylabel("singular value")
plt.title("Spectrum of the photograph")
plt.grid(alpha=0.3)
plt.show()
```



- at 20% of the storage the rank-50 image is perfectly readable
- real codecs (JPEG) use a fixed cosine basis on small blocks instead of a data-dependent one, but the principle is the same
- the same low-rank idea powers latent semantic analysis of term–document matrices (Deerwester et al., 1990)

3.2 Anomaly detection by reconstruction error

If normal data live near a k-dimensional subspace, then the reconstruction error $e(x) = \|x - \hat{x}\|^2$ with $\hat{x} = \mu + V_k V_k^\top (x - \mu)$ is small for normal points and large for points off the subspace. This simple rule is used for sensor monitoring, fraud screening and quality control, and its nonlinear version (the autoencoder) returns in Week 12.

Experiment: fit PCA (90% variance) on training digits, then score held-out digits and their **mirror images** — unusual inputs that look like plausible strokes.

In [38]:
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

X_tr, X_te, y_tr, y_te = train_test_split(X / 16.0, y, test_size=0.3, random_sta
```

```
pca90 = PCA(n_components=0.9).fit(X_tr)

def reconstruction_error(A):
    return np.sum((A - pca90.inverse_transform(pca90.transform(A))) ** 2, axis=1

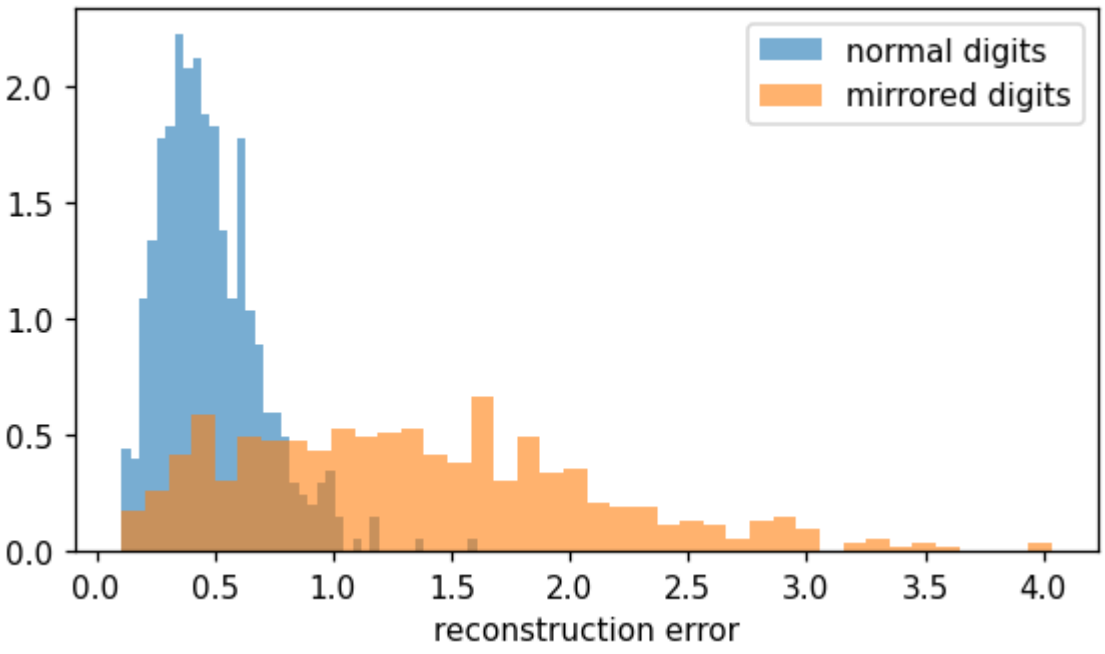
X_mirror = X_te.reshape(-1, 8, 8)[:, :, ::-1].reshape(len(X_te), -1)
e_normal, e_mirror = reconstruction_error(X_te), reconstruction_error(X_mirror)

def auc(normal, anomalous):
    labels = np.r_[np.zeros(len(normal)), np.ones(len(anomalous))]
    return float(roc_auc_score(labels, np.r_[normal, anomalous]))

per_class = {int(c): round(auc(e_normal[y_te == c], e_mirror[y_te == c]), 3) for
{"components": int(pca90.n_components_), "overall AUC": round(auc(e_normal, e_mi
 "AUC per digit": per_class}
```

Out[38]:
```
{'components': 21,
 'overall AUC': 0.876,
 'AUC per digit': {0: 0.795,
  1: 0.611,
  2: 0.968,
  3: 0.974,
  4: 0.975,
  5: 0.988,
  6: 0.986,
  7: 0.993,
  8: 0.739,
  9: 0.945}}
```

In [39]:
```
fig, ax = plt.subplots(figsize=(6, 3.2))
ax.hist(e_normal, bins=40, alpha=0.6, density=True, label="normal digits")
ax.hist(e_mirror, bins=40, alpha=0.6, density=True, label="mirrored digits")
ax.set_xlabel("reconstruction error")
ax.legend()
plt.show()
```



The detector works well overall, but look at the per-class AUC: it is weakest for **0, 1 and 8** — the digits that are (almost) symmetric under a left–right flip. Their mirror images *are*

normal digits, so nothing is anomalous to detect. A useful reminder that "anomaly" is always relative to what the representation can distinguish.

3.3 How not to read a t-SNE (or UMAP) plot

Neighbourhood embeddings are among the most widely published figures in science — single-cell transcriptomics papers routinely show UMAP maps of cell types, and embedding projectors are the standard way to inspect word or image embeddings. They are also easy to over-interpret (Wattenberg, Viégas & Johnson, 2016; Chari & Pachter, 2023).

A controlled synthetic test in $\mathbb{R}^{50}$: three Gaussian clusters of 200 points each.

- A and B are **close** (centres 6 apart), C is **far** (60 apart from A).
- A and C have spread 1; B is **ten times tighter** (spread 0.1).

In [40]:
```
dim = 50
e1 = np.eye(dim)[0]
A_cl = rng.normal(0.0, 1.0, (200, dim))
B_cl = 6.0 * e1 + rng.normal(0.0, 0.1, (200, dim))
C_cl = 60.0 * e1 + rng.normal(0.0, 1.0, (200, dim))
X_syn = np.vstack([A_cl, B_cl, C_cl])
lab_syn = np.repeat([0, 1, 2], 200)

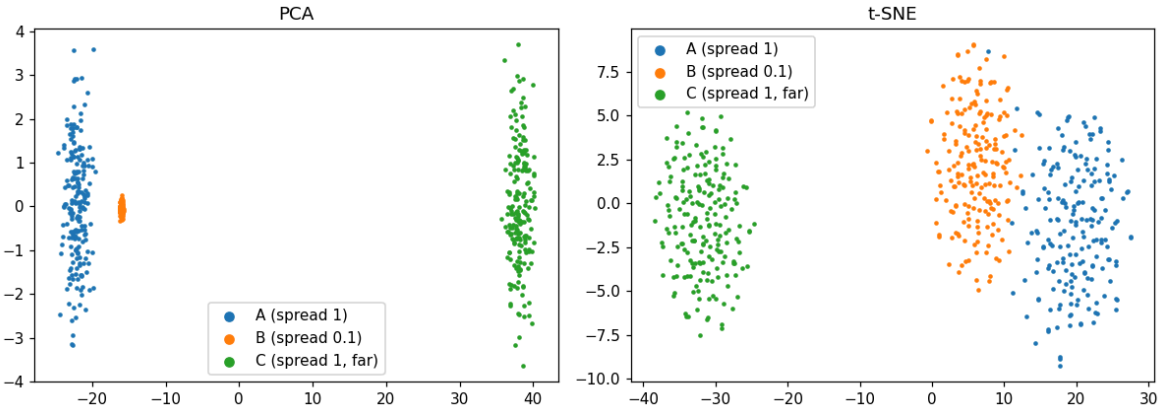
E_pca = PCA(n_components=2).fit_transform(X_syn)
E_tsne = TSNE(perplexity=30, init="pca", random_state=0).fit_transform(X_syn)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, E, name in zip(axes, [E_pca, E_tsne], ["PCA", "t-SNE"]):
    for c, nm in zip(range(3), ["A (spread 1)", "B (spread 0.1)", "C (spread 1,
        ax.scatter(E[lab_syn == c, 0], E[lab_syn == c, 1], s=4, label=nm)
    ax.set_title(name)
    ax.legend(markerscale=3)
plt.tight_layout()
plt.show()

def spread(E, c):
    P = E[lab_syn == c]
    return float(np.linalg.norm(P - P.mean(axis=0), axis=1).mean())

def gap(E, c1, c2):
    return float(np.linalg.norm(E[lab_syn == c1].mean(axis=0) - E[lab_syn == c2]

{"t-SNE spread A / spread B": round(spread(E_tsne, 0) / spread(E_tsne, 1), 2),
 "t-SNE gap A-C / gap A-B": round(gap(E_tsne, 0, 2) / gap(E_tsne, 0, 1), 2),
 "true spread ratio": 10.0, "true gap ratio": 10.0}
```



Out[40]:
```
{'t-SNE spread A / spread B': 1.2,
 't-SNE gap A-C / gap A-B': 3.66,
 'true spread ratio': 10.0,
 'true gap ratio': 10.0}
```

PCA keeps the true geometry (the far cluster is far; B is a dot) but crushes all within-cluster detail. t-SNE shows three clean clusters — and silently equalises their sizes and compresses the tenfold difference in separation. Rules of thumb:

1. Cluster **membership** and local neighbourhoods are meaningful.
2. Cluster **sizes** and **distances between clusters** generally are not.
3. Apparent structure can depend on perplexity / `n_neighbors` and on the seed — vary them.
4. Quantitative conclusions belong to the original space (or to a linear map such as PCA), not to the picture.

Where these tools appear in practice.

- *Genomics.* The first two principal components of ~200 000 genetic markers of Europeans reproduce the map of Europe (Novembre et al., *Nature* 2008); PCA is the standard correction for population structure in association studies.
- *Faces and images.* Eigenfaces (Turk & Pentland, 1991) were the first practical face-recognition system: PCA on aligned face images, nearest neighbour on the scores.
- *Search and recommendation.* Random projections and low-rank factorisations make similarity search over billions of items affordable (Week 13).
- *Model inspection.* t-SNE / UMAP maps of learned embeddings are the first diagnostic for text, image and audio models alike — we will use them in every block of the course.

Summary

- **Similarity** is a choice: dot product, cosine and Euclidean distance answer different questions; after normalisation cosine and Euclidean rank identically; Mahalanobis is Euclidean after whitening.
- **High dimension** concentrates volume near the boundary, makes distances nearly equal and random directions nearly orthogonal; nearest neighbours in noise are meaningless.
- **Real data have low intrinsic dimension**, and dimensionality reduction exploits it.

- **Random projections** (JL) preserve all distances up to $1 \pm \varepsilon$ with $k = O(\varepsilon^{-2} \log n)$, independent of d.
- **PCA** = variance maximisation = minimum reconstruction error = truncated SVD of the centred data (Eckart–Young). Randomised SVD scales it, using mostly matrix products on CPU / CUDA / MPS.
- **t-SNE / UMAP** preserve neighbourhoods, not global geometry; read them accordingly.

In [ ]: